Task 21: Running Stages in Parallel With Jenkins Workflow / Pipeline

1. Objective

To execute multiple Jenkins pipeline stages simultaneously using the parallel block, reducing total build time by running independent tasks concurrently.

2. Purpose

In a sequential pipeline, every stage waits for the previous one to finish. This is correct when stages have dependencies (you must build before you test), but wasteful when stages are independent. For example:

- **Unit Tests** and **Integration Tests** — independent of each other
- **Static Analysis** and **Security Scan** — independent of each other
- **Linting** and **Compilation** — can run together
- **Multi-platform builds** — Linux, Windows, macOS can build in parallel
- **Multi-region deployments** — deploy to us-east, eu-west, ap-south simultaneously

Running these stages in parallel:

- **Reduces total build time** — three 5-minute stages run in 5 minutes total instead of 15
- **Speeds up feedback** — developers learn about failures faster
- **Improves resource utilization** — Jenkins executors on multiple nodes work concurrently
- **Enables earlier error detection** — multiple failure sources discovered in one run

Without parallelism, CI/CD pipelines become bottlenecks, slowing down development velocity. This task demonstrates three types of stages (sequential, parallel, conditional) and shows how to define parallel blocks that execute concurrently.

3. Prerequisites

Item	Requirement
Jenkins Version	2.580.1
Agent	Built-in Node (external agents offline) — agent any uses the controller
Job Type	Pipeline (Declarative)
Executors	At least 2 available executors on the node
Access	Admin or job-configure permission

Note on Executors: If the Built-in Node has only 1 executor, parallel stages will not actually run simultaneously — they will queue. Check **Manage Jenkins → Nodes → Built-in Node → Configure** and set **# of executors** to 2 or more for true parallelism. Alternatively, external agents provide additional executors

4. Plugins Needed

Feature Plugin	Required
parallel block Pipeline	None — built into Jenkins
parallel step (scripted)	None — built into Pipeline
Stage View	Pipeline: Stage View plugin (usually pre-installed)
Blue Ocean (visualization)	Blue Ocean plugin (optional)

5. Concepts Reference

5.1 Three Types of Stages

type	Behavior	Example
Sequential	Runs one after another	Build → Test → Deploy
Parallel	Runs simultaneously and independently	Unit Tests ‖ Integration Tests
Conditional	Runs only if a condition is met	Deploy only on main branch

5.3 The parallel Block Syntax

Declarative (inside a stage):

```
groovy
stage('Tests') {
   parallel {
      stage('Unit Tests') { steps { ... } }
      stage('Integration Tests') { steps { ... } }
   }
}
```

Scripted (as a step):

```
groovy
parallel(
   "Unit Tests": { ... },
   "Integration Tests": { ... }
)
```

5.4 When to Use Parallel

Use Parallel	Don't Use Parallel
Independent tests	Tests sharing a database
Multi-platform builds	Steps using the same workspace file
Static analysis + linting	Steps requiring the previous step's output
Deployment to multiple regions	Deployment to a single environment (order matters)
Downloading multiple dependencies	Steps with shared global state

6. Step-by-Step Procedures

Step 0: Check Available Executors

Step 0.1: Go to **Manage Jenkins → Nodes**

Step 0.2: Click **Built-in Node**

Step 0.3: Check the **# of executors** value. It should be **2 or more** for parallel stages to run simultaneously.

If it's 1: Click **Configure** → Set **# of executors** to 2 or 3 → **Save**

Step 0.4: Go to the dashboard and observe the **Build Executor Status** panel. You should see multiple executor slots on the Built-in Node.

Step 0 Complete.

Method 1: Basic Parallel Block in Declarative Pipeline

Step 1.1: Create job Parallel-Stages-Demo (Pipeline).

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                sh 'echo "Building the application"'
            }
        }
        stage('Test') {
            parallel {
                stage('Unit Tests') {
                    steps {
                        sh 'sleep 5s'
                        sh 'echo "Running unit tests"'
                    }
                }
                stage('Integration Tests') {
                    steps {
                        sh 'sleep 3s'
                        sh 'echo "Running integration tests"'
                    }
                }
            }
        }
        stage('Deploy') {
            steps {
                sh 'echo "Deploying the application"'
            }
        }
    }
}
```

Step 1.5: Measure total duration

- Sequential would take 5 + 3 = 8 seconds
- Parallel takes 5 seconds (the longer branch)
- Check the **Duration** column in Build History to confirm

Why it works:

- The parallel { } block defines nested stages that run simultaneously
- Each nested stage gets its own executor
- The Test stage does not complete until both parallel branches finish
- Subsequent stages (Deploy) wait for the entire parallel block

Method 2: Dynamic Parallel Stages (From a List)

```
def testSuites = ['Unit', 'Integration', 'E2E', 'Smoke']

pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo "Building the application"
            }
        }
        stage('Dynamic Parallel Tests') {
            steps {
                script {
                    def parallelStages = testSuites.collectEntries { suite ->
                        ["${suite} Tests": {
                            stage("${suite} Tests") {
                                sh "echo 'Running ${suite} tests'"
                                sh 'sleep 3s'
                                sh "echo '${suite} tests completed'"
                            }
                        }]
                    }
                    parallel parallelStages
                }
            }
        }
        stage('Deploy') {
            steps {
                echo "Deploying the application"
            }
        }
    }
}
```

Why it works:

- testSuites.collectEntries { suite -> [...] } builds a Groovy map where:
    - **Key** = stage name (e.g., Unit Tests)
    - **Value** = closure containing the stage's steps
- parallel parallelStages runs all entries in the map simultaneously
- To add a new test suite, just add it to the testSuites list — no other changes needed

Executor limitation: The Built-in Node has 2 executors by default. With 4 parallel branches, 2 will start immediately and the other 2 will wait in the queue. To run all 4 truly in parallel, increase the Built-in Node's executors to 4+ (**Manage Jenkins → Nodes → Built-in Node → Configure → # of executors**).

Method 3: Dynamic Parallel Stages from a JSON Parameter

Step 3.1: Add a Multiline String Parameter

1. Scroll to **General** → check **This project is parameterized**
2. Click **Add Parameter** → **Multiline String Parameter**
3. **Name:** testMatrix
4. **Default Value:**

```
{
  "tests": [
    {"name": "Unit", "duration": 3},
    {"name": "Integration", "duration": 5},
    {"name": "E2E", "duration": 4}
  ]
}
```

Step 3.2: Update the pipeline script:

```
pipeline {
    agent any
    parameters {
        text(
            name: 'testMatrix',
            defaultValue:
'''{"tests":[{"name":"Unit","duration":3},{"name":"Integration","duration":5},{"name":"E2E","duratio
n":4}]}''',
            description: 'JSON defining test suites'
        )
    }
    stages {
        stage('Build') {
            steps {
                echo "Building the application"
            }
        }
        stage('Parallel Tests from JSON') {
            steps {
                script {
                    def config = readJSON text: params.testMatrix
                    def parallelStages = config.tests.collectEntries { test ->
                        ["${test.name} Tests": {
                            stage("${test.name} Tests") {
                                sh "echo 'Running ${test.name} tests (${test.duration}s)'"
                                sh "sleep ${test.duration}s"
                                sh "echo '${test.name} tests completed'"
                            }
                        }]
                    }
                    parallel parallelStages
                }
            }
        }
```

```
    stage('Deploy') {
      steps {
        echo "Deploying the application"
      }
    }
  }
}
```

Why this is powerful:

- Test configuration is **externalized** to a JSON parameter
- Users can change which tests run by editing the JSON before triggering the build
- New tests are added by editing JSON — no Jenkinsfile changes

Key behavior: When Integration Tests fails, the E2E Tests branch is aborted immediately rather than waiting to finish. This saves time by not running work that will be discarded.

Alternative: Use catchError or warnError (Task 3) inside parallel branches to control failure behavior.

Method 5: Scripted Pipeline with Parallel Step

```
node {
  stage('Build') {
    echo "Building the application"
  }

  stage('Parallel Tests') {
    def branches = [
      'Unit Tests': {
        echo "Running unit tests..."
        sleep 3
        echo "Unit tests completed"
      },
      'Integration Tests': {
        echo "Running integration tests..."
        sleep 4
        echo "Integration tests completed"
      },
      'Lint': {
        echo "Running linter..."
        sleep 2
        echo "Linting completed"
      }
    ]
    parallel branches
  }

  stage('Deploy') {
    echo "Deploying the application"
  }
}
```

Why it works:

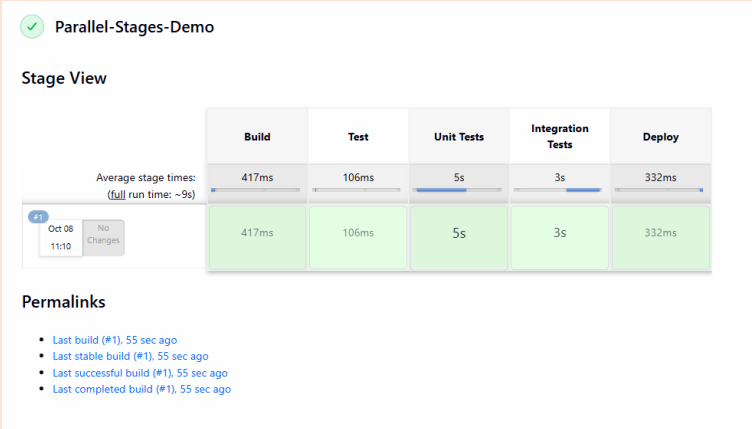
- In Scripted pipelines, the parallel step takes a map directly
- Each key is a branch name; each value is a closure with the branch's steps
- No parallel { } block syntax — just a Groovy map

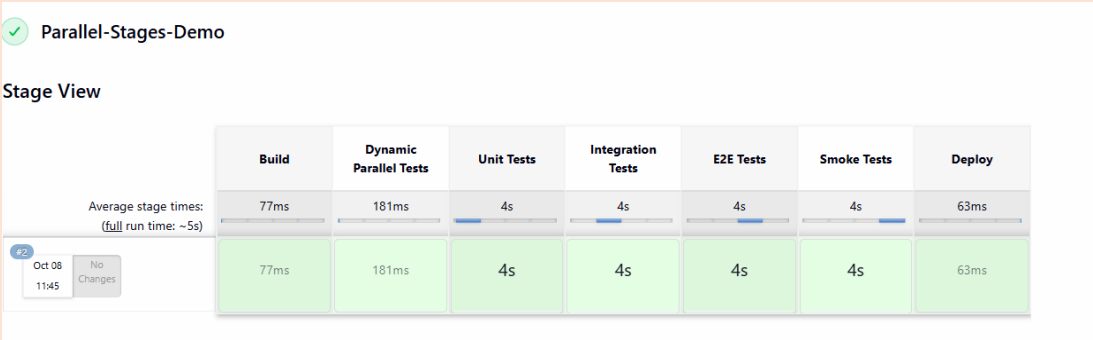
Verification

For each method, verify:

- **Method 1:** Two test stages appear side-by-side in Stage View; total duration ≈ 5s (longest branch)
- **Method 2:** Four parallel test stages; total duration ≈ 3s (longest branch)
- **Method 3:** Three test stages from JSON; each with own duration; total ≈ 5s
- **Method 4:** On failure, remaining branches are aborted (check timestamps)
- **Method 5:** Scripted pipeline shows three parallel branches

Implementation:





Pipeline Parallel-Stages-Demo

This build requires parameters:

testMatrix

```
{
  "tests": [
    {"name": "Unit", "duration": 3},
    {"name": "Integration", "duration": 5},
    {"name": "E2E", "duration": 4}
  ]
}
```

Build Cancel

Stage View

	Build	Parallel Tests from JSON	Unit Tests	Integration Tests	E2E Tests	Deploy
Average stage times: (full run time: ~6s)	56ms	125ms	4s	6s	5s	46ms
#5 Oct 08 11:52 No Changes	56ms	125ms	4s	6s	5s	46ms

Pipeline Parallel-Stages-Demo

This build requires parameters:

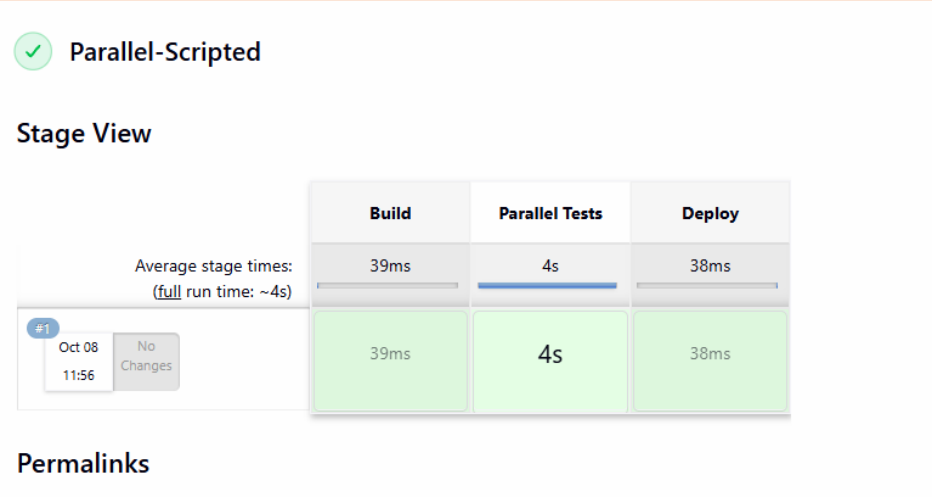
testMatrix

JSON defining test suites

```
{"tests":[{"name":"Unit","duration":3},{"name":"Integration","duration":5},{"name":"E2E","duration":4}]}
```

Build Cancel

```
[Pipeline] sh
[Unit Tests] + sleep 3s
[Integration Tests] + sleep 2s
[E2E Tests] + sleep 8s
[Pipeline] sh
[Integration Tests] + exit 1
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
Failed in branch Integration Tests
[Unit Tests] Sending interrupt signal to process
[E2E Tests] Sending interrupt signal to process
[Unit Tests] Terminated
[Pipeline] }
[E2E Tests] Terminated
[Pipeline] // stage
[Pipeline] }
Failed in branch Unit Tests
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
Failed in branch E2E Tests
[Pipeline] // parallel
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
ERROR: script returned exit code 1
Finished: FAILURE
```



Conclusion

In this article, we explored how to run multiple stages in parallel with a Jenkins workflow or pipeline job. First, we looked at the basic concepts and execution of the Jenkins pipeline job and its stages. After that, we ran multiple parallel stages in a Jenkins pipeline job. Finally, we also explored the benefits of running stages in parallel.

Task 22: Get the Output of a Shell Command in a Jenkins Pipeline

1. Objective

To capture the output and exit status of shell commands executed inside a Jenkins pipeline into Groovy variables, so that the output can be used as input for subsequent steps, conditions, or logic.

2. Purpose

By default, Jenkins prints shell command output to the console and discards it. But many pipelines need to **use** that output:

- **Read a Git commit hash** to tag a Docker image
- **Get the current directory** to build dynamic paths
- **List files** and act on each one
- **Check a service version** before deploying
- **Extract a value** from a command's output and pass it to the next stage
- **Branch on the exit status** of a command
- **Trim whitespace** from outputs so comparisons work correctly

5. Concepts Reference

5.1 The sh Step Flags

Flag	Returns	Use Case
(no flag)	Nothing (prints to console)	Simple execution
returnStdout: true	Command's stdout as a string	Capture output into a variable
returnStatus: true	Exit code as an integer	Check success/failure

5.2 Command Substitution

Shell syntax $(command) runs command and replaces the expression with its output. Useful inside sh strings to compose commands:

groovy
sh "echo \$(ls)"
The \$ escapes the $ so Groovy doesn't substitute it — the shell evaluates $(ls) at runtime.

5.3 Common Returns

Return	Meaning
0	Success
Non-zero (1, 2, 127, etc.)	Failure

5.4 Why Trim?

Shell command output usually includes a trailing newline (\n). Forgetting to trim causes comparisons like "main\n" == "main" to fail. Always .trim() unless newlines are needed.

6. Step-by-Step Procedures

Step 0: Create the Job

1. Click **New Item** → Name: Shell-Output-Demo → Select **Pipeline** → **OK**

```
pipeline {
  agent any
  stages {
    stage('Capture Output') {
      steps {
        script {
          def output = sh(returnStdout: true, script: 'pwd')
          echo "Current directory: ${output}"
        }
      }
    }
  }
}
```

Why it works:

- returnStdout: true tells the sh step to return the command's standard output
- The output is stored in the output variable as a **Groovy String**
- echo prints the variable's value

Notes:

- The output includes a trailing \n. Use .trim() to remove it (see Method 3)
- The shell command runs on the current node's workspace directory

Method 2: Using Shell Command Substitution

```
pipeline {
  agent any
  stages {
    stage('Command Substitution') {
      steps {
        script {
          def output = sh(
            script: "echo \$(ls)",
            returnStdout: true
          )
          echo "Files in workspace: ${output}"
        }
      }
    }
  }
}
```

Why it works:

- \$(ls) — the \$ escapes the $ so Groovy passes it to the shell literally
- The shell evaluates $(ls) — running ls and substituting its output
- echo prints the substitution result — which becomes the output of the whole sh step
- returnStdout: true captures that output into output

Note: This is a shell-only trick. Prefer returnStdout directly with the command you want, unless you specifically need to compose inside a shell string.

Method 3: Trimming Whitespace with .trim()

```
pipeline {
    agent any
    stages {
        stage('Trim Output') {
            steps {
                script {
                    def raw = sh(returnStdout: true, script: 'echo "    hello    "')
                    echo "Raw: '${raw}'"

                    def trimmed = sh(returnStdout: true, script: 'echo "    hello    "').trim()
                    echo "Trimmed: '${trimmed}'"
                }
            }
        }
    }
}
```

Why it works:

- sh returns the entire stdout including leading spaces and trailing newlines
- .trim() removes all leading and trailing whitespace characters (spaces, tabs, newlines)
- The single quotes around ${raw} in echo make whitespace visible in the console

When to use:

- Comparing output to expected values (if (output.trim() == 'main'))
- Using output in filenames, URLs, or image tags
- Preventing extra spaces in concatenated strings

Method 4: Capturing Exit Status with returnStatus: true

```
pipeline {
    agent any
    stages {
        stage('Check Exit Status') {
            steps {
                script {
                    def status1 = sh(returnStatus: true, script: 'ls /tmp')
                    echo "ls /tmp exit status: ${status1}"

                    def status2 = sh(returnStatus: true, script: 'ls /nonexistent-directory')
                    echo "ls /nonexistent-directory exit status: ${status2}"

                    if (status2 != 0) {
                        echo "❌ Command failed with status ${status2}"
                    } else {
                        echo "☑ Command succeeded"
                    }
                }
            }
```

```
    }
  }
}
```

Step 4.3: Observe the console output:

text

ls /tmp exit status: 0

ls /nonexistent-directory exit status: 2

❌ Command failed with status 2

Why it works:

- returnStatus: true returns the exit code instead of stdout
- Non-zero exit codes indicate failure
- The if block branches based on the result
- **Important:** When returnStatus: true is used, the sh step does NOT fail the pipeline even if the command fails

Common exit codes:

Code	Meaning
0	Success
1	General error
2	Misuse of shell builtins, or missing file/directory
126	Command found but not executable
127	Command not found
128+	Signal-related termination

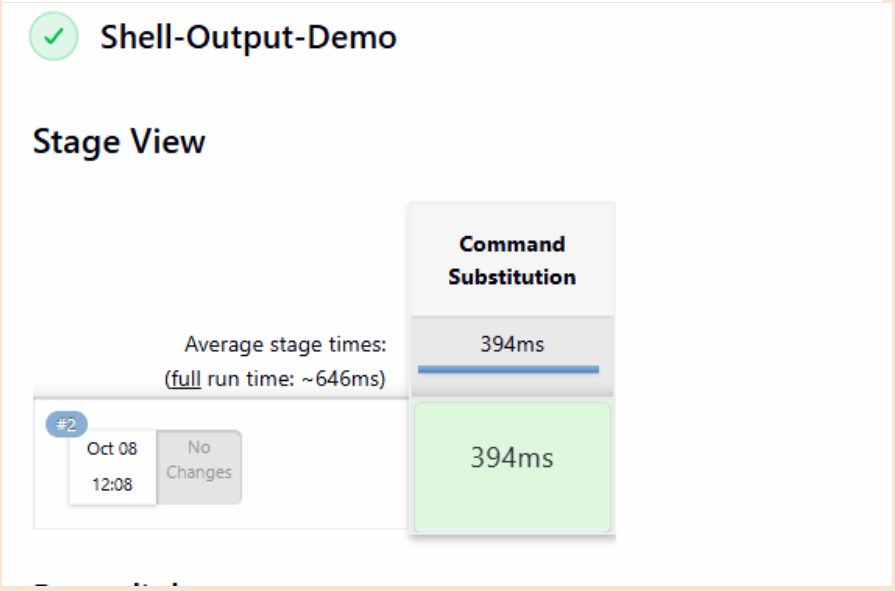
Method 5: Combining Stdout and Status

```
pipeline {
  agent any
  stages {
    stage('Stdout + Status Combined') {
      steps {
        script {
          def result = sh(
            returnStdout: true,
            script: 'ls /tmp && echo "STATUS:$?"'
          )
          echo "Combined output:"
          echo result
        }
      }
    }
  }
}
```

Implementation:

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Shell-Output-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Capture Output)
[Pipeline] script
[Pipeline] {
[Pipeline] sh
+ pwd
[Pipeline] echo
Current directory: /var/lib/jenkins/workspace/Shell-Output-Demo

[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Shell-Output-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Trim Output)
[Pipeline] script
[Pipeline] {
[Pipeline] sh
+ echo '    hello    '
[Pipeline] echo
Raw: '    hello
'
[Pipeline] sh
+ echo '    hello    '
[Pipeline] echo
Trimmed: 'hello'
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

```
[Pipeline] stage
[Pipeline] { (Check Exit Status)
[Pipeline] script
[Pipeline] {
[Pipeline] sh
+ ls /tmp
hsperfdata_jenkins
jetty-0_0_0_0-8080-war-_-any-13904897320690336223
systemd-private-ab7d386dec9f4177b89e043522176e8e-chronyd.service-aKBZSe
systemd-private-ab7d386dec9f4177b89e043522176e8e-dbus-broker.service-3MoHOv
systemd-private-ab7d386dec9f4177b89e043522176e8e-policy-routes@enp39s0.service-jkaHPQ
systemd-private-ab7d386dec9f4177b89e043522176e8e-systemd-logind.service-47Smek
systemd-private-ab7d386dec9f4177b89e043522176e8e-systemd-resolved.service-PvRZej
winstone10080857502545332386.jar
[Pipeline] echo
ls /tmp exit status: 0
[Pipeline] sh
+ ls /nonexistent-directory
ls: cannot access '/nonexistent-directory': No such file or directory
[Pipeline] echo
ls /nonexistent-directory exit status: 2
[Pipeline] echo
❌ Command failed with status 2
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

```
Running on Jenkins in /var/lib/jenkins/workspace/Shell-Output-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Stdout + Status Combined)
[Pipeline] script
[Pipeline] {
[Pipeline] sh
+ ls /tmp
+ echo STATUS:0
[Pipeline] echo
Combined output:
[Pipeline] echo
hsperfdata_jenkins
jetty-0_0_0_0-8080-war-_-any-13904897320690336223
systemd-private-ab7d386dec9f4177b89e043522176e8e-chronyd.service-aKBZSe
systemd-private-ab7d386dec9f4177b89e043522176e8e-dbus-broker.service-3MoHOv
systemd-private-ab7d386dec9f4177b89e043522176e8e-policy-routes@enp39s0.service-jkaHPQ
systemd-private-ab7d386dec9f4177b89e043522176e8e-systemd-logind.service-47Smek
systemd-private-ab7d386dec9f4177b89e043522176e8e-systemd-resolved.service-PvRZej
winstone10080857502545332386.jar
STATUS:0

[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Conclusion

In this article, we discussed two different methods for capturing the output of a shell command executed in the Jenkins pipeline into a variable.

First, we looked into capturing the output using *sh returnStdout*. After that, we explored the same using command substitution. Both methods can capture the output of a shell command. Despite this, the *returnStdout* method of *sh* is easier to read and simpler than command substitution.

Task 23: Jenkins Pipeline – Change to Another Folder

1. Objective

To change the working directory inside a Jenkins pipeline using two methods: the built-in dir step (block-scoped directory change) and the sh step with the shell cd command.

2. Purpose

Jenkins pipelines usually run all steps from the job's workspace root. However, many real-world tasks require running commands from a specific subdirectory:

- **Running build scripts** located in a scripts/ subfolder
- **Executing npm/gradle/maven commands** in a project subdirectory
- **Handling monorepos** where each service has its own directory
- **Running tests** in a tests/ or spec/ folder
- **Accessing configuration files** that live in a config/ directory
- **Using the File Operations plugin** (from Task 17) which only works in the current directory
- **Isolating side effects** to a specific folder

Without changing the working directory, every command must use full paths or relative paths from the workspace root — which is verbose and error-prone. This task demonstrates the two standard ways to change directories: dir (declarative, block-scoped) and cd (shell-based, must be chained in a single sh call).

5. Concepts Reference

5.1 What Is a Working Directory?

The working directory (or current directory) is the folder where relative paths are resolved. When a Jenkins pipeline runs a shell command, it starts in the job's **workspace directory** (e.g., /var/lib/jenkins/workspace/Job-Name/).

5.2 Two Ways to Change Directory

Method	Scope	Return Behavior	Best For
dir('path') { ... }	The block only	Automatically reverts after block ends	Declarative pipelines, multiple commands
sh 'cd path && ...'	The single shell command	Does NOT persist across sh calls	Simple one-off commands

5.3 Critical Difference: dir vs sh 'cd'

sh 'cd somedir' **alone does NOT change the working directory for subsequent steps.** Each sh step starts a **new shell process** from the workspace root. The cd only affects the current shell invocation:

groovy

```
sh 'cd scripts'      // Changes directory in this shell, then the shell exits
sh 'pwd'             // Still in workspace root!
```

To persist a directory change across commands, either:

- Use dir('scripts') { ... } wrapping the commands, **or**
- Chain commands in a single sh call: sh 'cd scripts && pwd && ls'

5.4 Absolute vs Relative Paths

- **Relative paths** — resolved against the current working directory
- **Absolute paths** — always start with /
- dir('scripts') works for subdirectories
- dir('/var/lib/jenkins/workspace/Job/scripts') works for anywhere (with permissions)

6. Step-by-Step Procedures

Step 0: Create the Job and Test Directories

Step 0.1: Create a new pipeline job:

1. Click **New Item** → Name: Change-Dir-Demo → Select **Pipeline** → **OK**

```
pipeline {
  agent any
  stages {
    stage('Setup Directories') {
      steps {
        sh '''
        mkdir -p scripts
        echo "This is a script file" > scripts/run.sh
```

```
            echo "Project configuration" > scripts/config.txt
            ls -la scripts/
            '''
        }
    }
  }
}
```

Method 1: Using the dir Step

```
pipeline {
  agent any
  stages {
    stage('Setup') {
      steps {
        sh '''
        mkdir -p scripts
        echo "echo hello from script" > scripts/run.sh
        echo "config=value" > scripts/config.txt
        '''
      }
    }
    stage('Change Directory with dir') {
      steps {
        script {
          echo "Before dir: ${pwd()}"
        }
        dir('scripts') {
          script {
            echo "Inside dir: ${pwd()}"
          }
          sh 'ls -la'
          sh 'cat config.txt'
          sh 'cat run.sh'
        }
        script {
          echo "After dir: ${pwd()}"
        }
      }
    }
  }
}
```

Why it works:

- dir('scripts') { ... } switches the working directory for the duration of the block
- All sh and other steps inside the block run in scripts/
- After the block ends, the working directory automatically reverts to the original workspace — visible from the After dir log line

Why this is the recommended method:

- Block-scoped: no risk of forgetting to change back

- Works with any Jenkins step (sh, readFile, writeFile, fileOperations, etc.)
- Readable and declarative
- Works for absolute paths too — dir('/var/lib/jenkins/...')

Method 2: Using sh 'cd ...' — The Correct Way (Chained Commands)

```
pipeline {
  agent any
  stages {
    stage('Setup') {
      steps {
        sh '''
        mkdir -p scripts
        echo "config=value" > scripts/config.txt
        '''
      }
    }
    stage('Change Directory with sh cd') {
      steps {
        script {
          echo "Before cd: ${pwd()}"
        }
        sh '''
        cd scripts
        pwd
        ls -la
        cat config.txt
        '''
        script {
          echo "After sh block: ${pwd()}"
        }
      }
    }
  }
}
```

Why it works:

- Inside the **single** sh block, all commands run in the same shell
- The cd scripts changes the directory for the rest of that shell session
- Subsequent commands (pwd, ls, cat) see the new directory
- When the sh block finishes, the shell exits, and the next step starts fresh in the original workspace

Method 3: Using dir with an Absolute Path

Why this is useful: You can access, inspect, or manipulate files anywhere the jenkins user has permission — not just inside the workspace.

Note: The Groovy Sandbox may block dir('/absolute/path'). If that happens:

1. Go to the job → **Configure**
2. Scroll to Pipeline section → **uncheck** "Use Groovy Sandbox"
3. Save and rerun

```
pipeline {
  agent any
  stages {
    stage('Change to Absolute Path') {
      steps {
        script {
          echo "Before: ${pwd()}"
        }
        dir('/var/lib/jenkins') {
          script {
            echo "Inside: ${pwd()}"
          }
          sh 'ls -la | head -5'
        }
        script {
          echo "After: ${pwd()}"
        }
      }
    }
  }
}
```

Why this is useful: You can access, inspect, or manipulate files anywhere the jenkins user has permission — not just inside the workspace.

Note: The Groovy Sandbox may block dir('/absolute/path'). If that happens:

1. Go to the job → **Configure**
2. Scroll to Pipeline section → **uncheck** "Use Groovy Sandbox"
3. Save and rerun

Method 4: Chaining Multiple dir Blocks (Nested Directories)

```
pipeline {
  agent any
  stages {
    stage('Setup Nested Directories') {
      steps {
        sh '''
        mkdir -p project/src project/tests
        echo "main code" > project/src/main.txt
        echo "test code" > project/tests/test.txt
        '''
      }
    }
    stage('Nested dir Blocks') {
      steps {
        dir('project') {
          script { echo "Project dir: ${pwd()}" }
          sh 'ls -la'

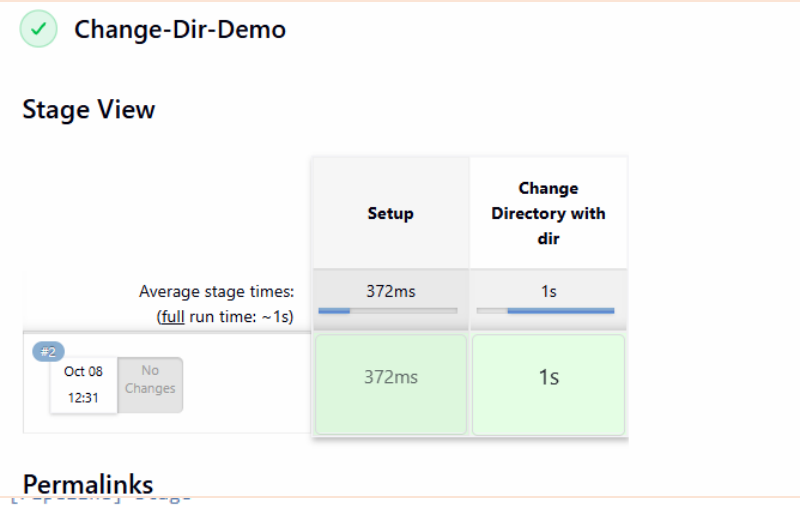
          dir('src') {
```

```
                script { echo "Src dir: ${pwd()}" }
                sh 'cat main.txt'
              }

              dir('tests') {
                script { echo "Tests dir: ${pwd()}" }
                sh 'cat test.txt'
              }
            }
            script { echo "Back at: ${pwd()}" }
          }
        }
    }
}
```

Best Practices

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on jenkins-agent-01 in /home/jenkins/workspace/Change-Dir-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Setup Directories)
[Pipeline] sh
+ mkdir -p scripts
+ echo 'This is a script file'
+ echo 'Project configuration'
+ ls -la scripts/
total 8
drwxr-xr-x. 2 jenkins jenkins 38 Oct  8 06:59 .
drwxr-xr-x. 3 jenkins jenkins 21 Oct  8 06:59 ..
-rw-r--r--. 1 jenkins jenkins 22 Oct  8 06:59 config.txt
-rw-r--r--. 1 jenkins jenkins 22 Oct  8 06:59 run.sh
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```




```
[Pipeline] { (Change Directory with sh cd)
[Pipeline] script
[Pipeline] {
[Pipeline] pwd
[Pipeline] echo
Before cd: /home/jenkins/workspace/Change-Dir-Demo
[Pipeline] }
[Pipeline] // script
[Pipeline] sh
+ cd scripts
+ pwd
/home/jenkins/workspace/Change-Dir-Demo/scripts
+ ls -la
total 8
drwxr-xr-x. 2 jenkins jenkins 38 Oct  8 06:59 .
drwxr-xr-x. 4 jenkins jenkins 40 Oct  8 07:01 ..
-rw-r--r--. 1 jenkins jenkins 13 Oct  8 07:04 config.txt
-rw-r--r--. 1 jenkins jenkins 23 Oct  8 07:01 run.sh
+ cat config.txt
config=value
[Pipeline] script
[Pipeline] {
[Pipeline] pwd
[Pipeline] echo
After sh block: /home/jenkins/workspace/Change-Dir-Demo
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Conclusion

In this article, we learned how to change the working directory in a Jenkins pipeline. First, we looked at changing the directory using the *dir* step. After that, we explored the same using the *cd* command with the *sh* step.

Task 24: Fixing the "No Such DSL Method" Error in Jenkins Pipeline

1. Objective

To diagnose and fix the No such DSL method error in Jenkins Pipelines by identifying the four common causes: typos, missing plugins, outdated Jenkins, and outdated syntax.

2. Purpose

The No such DSL method error is one of the most common and confusing errors Jenkins developers encounter. It occurs when Jenkins' Pipeline DSL (Domain Specific Language) does not recognize a method used in a Jenkinsfile. Without understanding its causes, developers waste hours trying to fix a simple typo or chase a phantom bug.

Understanding and fixing this error is essential for:

- **Faster debugging** — recognize the error instantly and go to the likely cause
- **Correct plugin management** — know when a plugin is missing vs. when the syntax is wrong
- **Version awareness** — recognize when an old Jenkinsfile uses deprecated methods
- **Production reliability** — avoid deploying pipelines that fail on start
- **Team productivity** — standard debugging patterns reduce friction

Common scenarios where this error appears:

- Typing mvn 'clean instal' (typo in the Maven goal)
- Using readJSON without the Pipeline Utility Steps plugin
- Using a deprecated method like fileExists (older syntax) in a modern Jenkins
- Using dockerImage in a plugin that was renamed

5. Concepts Reference

5.1 What Is the Pipeline DSL?

The Jenkins Pipeline DSL (Domain Specific Language) is the set of methods and constructs that Jenkins recognizes inside a Jenkinsfile. Examples:

- sh, bat — execute shell commands
- echo, error — control output and failure
- stage, steps, parallel — pipeline structure
- fileExists, readFile, readJSON — file operations
- mvn, gradle — build tool wrappers

If a method is not in the DSL, Jenkins throws:

No such DSL method 'method_name' found among steps [...]

5.2 The Four Common Causes:

#	Cause	Example	Fix
1	Typo	mvn 'clean instal'	Correct the typo: mvn 'clean install'
2	Missing plugin	readJSON without Pipeline Utility Steps	Install the plugin
3	Outdated Jenkins	Modern method on old Jenkins	Upgrade Jenkins
4	Outdated syntax	Deprecated method in Jenkinsfile	Use the modernmethod

5.3 Error Message Anatomy

The full error message usually includes:

text

java.lang.NoSuchMethodError: No such DSL method 'X' found among steps

[... long list of known methods ...]

- No such DSL method 'X' — Jenkins doesn't recognize X
- found among steps [...] — a list of valid methods (useful for verification)
- **Line number and column** — where the error occurred in the Jenkinsfile

5.4 Debugging with -X

Adding -X to the Jenkins startup (or to a specific script via sh 'bash -x script.sh') enables verbose output. This is useful when the error is subtle.

6. Step-by-Step Procedures

Step 0: Create the Job

1. Click **New Item** → Name: No-DSL-Method-Demo → Select **Pipeline** → **OK**

Method 1: Fix a Typo Error

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        mvn 'clean instal'
      }
    }
  }
}
```

Step 1.3: Observe the error (or a related failure):

```
[Pipeline] mvn
[Pipeline] End of Pipeline
ERROR: No such DSL method 'mvn' found among steps
```

OR (if Maven is not installed on the Jenkins host):

text

```
ERROR: mvn: command not found
```

Step 1.4: Fix the typo

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        sh 'mvn clean install'
      }
    }
  }
}
```

Step 1.6: Observe the output — no DSL error. If Maven is not installed on the Jenkins host, you'll see mvn: command not found, which is a **separate** issue (install Maven on the host).

Key insight: The mvn step is not part of the Jenkins Pipeline DSL. To run Maven, either:

- Install the **Maven Integration Plugin** (adds the mvn step), OR
- Use sh 'mvn ...' to run Maven as a shell command

Most common typo pitfalls:

Typo	Fix
mvn 'clean instal'	mvn 'clean install' (or sh 'mvn clean install')
ech 'hello'	echo 'hello'
scm 'git'	checkout scm
stage { } (missing quotes)	stage('Build') { }
setps { }	steps { }

Method 2: Fix a Missing Plugin Error

```
pipeline {
   agent any
   stages {
      stage('Parse JSON') {
         steps {
            script {
               def data = readJSON text: '{"key":"value"}'
               echo "Parsed: ${data.key}"
            }
         }
      }
   }
}
```

Step 2.2: Save → Build Now

Step 2.3: Observe the error:

text

java.lang.NoSuchMethodError: No such DSL method 'readJSON' found among steps

Step 2.4: Install the missing plugin

1. Go to **Manage Jenkins → Plugins → Available**
2. Search Pipeline Utility Steps
3. Install and restart if required

Step 2.5: Save the job again → Build Now

Step 2.6: Observe the output:

text

Parsed: value

Finished: SUCCESS

Why it works: readJSON is part of the **Pipeline Utility Steps** plugin, not Jenkins core. Once the plugin is installed, the DSL recognizes the method.

Common missing-plugin scenarios:

Method	Plugin Required
readJSON, writeJSON, readYaml, writeYaml	Pipeline Utility Steps
findFiles, readCSV	Pipeline Utility Steps
docker, dockerImage	Docker Pipeline
junit, publishHTML	JUnit / HTML Publisher
slackSend	Slack Notification
checkout scm	Git (usually pre-installed)

Diagnosis tip: If Jenkins says "No such DSL method", search the method name on https://plugins.jenkins.io to find which plugin provides it.

Method 3: Fix an Outdated Jenkins Version

Step 3.1: Some methods are only available in newer Jenkins versions. If you use a modern method on old Jenkins, you get the DSL error. There's no code example that recreates this safely — the fix is to upgrade Jenkins.

Step 3.2: Check your Jenkins version

1. Go to **Manage Jenkins → System Information** (or the bottom-right of the Dashboard)
2. Note the version (e.g., 2.580.1)

Step 3.3: Upgrade Jenkins

Choose one of:

- **Via Jenkins UI:** Manage Jenkins → System → Look for "Update" notification and click "Upgrade"
- **Via WAR file:** Download the latest jenkins.war from https://www.jenkins.io/download/ and replace the existing one
- **Via Docker (if running in a container):** Pull the newer image and recreate the container

Step 3.4: Restart Jenkins and rerun the pipeline

Why this matters: New Jenkins versions often add DSL methods. Running old Jenkins with a Jenkinsfile written for newer versions causes the "No such DSL method" error.

Verification:

```
pipeline {
  agent any
  stages {
    stage('Check Version') {
      steps {
        echo "Jenkins version: ${env.JENKINS_VERSION}"
      }
    }
  }
}
```

Method 4: Fix Outdated Syntax

Step 4.1: Older Jenkinsfiles used certain deprecated methods that modern Jenkins no longer

```
// Deprecated scripted pipeline style (still works, but old)
node('master') {
   stage 'Build'
   sh 'make'
}
```

Modern Jenkins prefers:

```
// Modern declarative style
pipeline {
   agent { label 'master' }
   stages {
      stage('Build') {
         steps {
            sh 'make'
         }
      }
   }
}
```

Step 4.3: Common outdated syntax patterns

Old Syntax	Modern Syntax
stage 'Build' (no block)	stage('Build') { }
node('label') { }	agent { label 'label' }
checkout scm	checkout scm (still valid)
parallel(branches)	parallel { stage('X') { } } (declarative)
withEnv([...]) { }	Same — still valid

Method 5: Use the Error Message to Find the Correct Method

Step 5.1: When the DSL error appears, look at the found among steps [...] list. It contains every valid method. For example:

```
No such DSL method 'readJsn' found among steps
[Pipeline] End of Pipeline
ERROR: No such DSL method 'readJsn' found among steps
[ ...
sh, bat, echo, error, parallel, fileExists, readFile, ...
]
```

Step 5.2: Scanning the list, we see readFile but not readJsn. The likely intended method was readJSON (if the plugin is installed) or readFile.

Step 5.3: Fix the method name and rerun.

Diagnostic workflow:

1. Read the exact method name from the error message
2. Check the list of valid methods in the console output
3. Look for a method with a similar name (likely a typo)

4. Search Jenkins documentation or https://plugins.jenkins.io for the method
5. Check if the plugin providing it is installed
6. Fix the name, install the plugin, or update the Jenkinsfile

Method 6: Enable Debug Output with -X

Step 6.1: For subtle errors, enable verbose output. On the Jenkins host:
bash
java -jar jenkins.war -X
Or, if Jenkins runs as a service, edit the service file to add -X to the Java command.

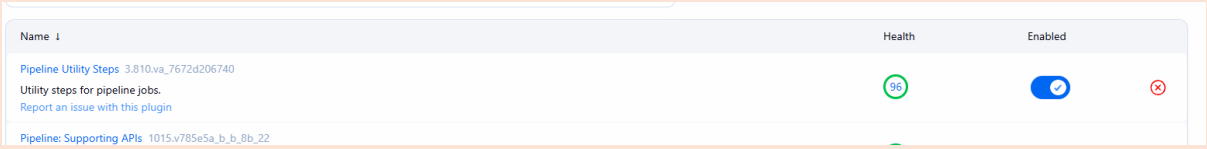
Step 6.2: Restart Jenkins and rerun the failing pipeline. The verbose log includes the DSL parser's view of your Jenkinsfile, which helps pinpoint the exact issue.
Alternative for shell scripts inside the pipeline:

groovy
sh 'bash -x script.sh'

This prints each shell command before executing it.
Implementation:

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on jenkins-agent-01 in /home/jenkins/workspace/No-DSL-Method-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] sh
+ mvn clean install
/home/jenkins/workspace/No-DSL-Method-Demo@tmp/durable-db81b872/script.sh.copy: line 1: mvn: command not found
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
ERROR: script returned exit code 127
Finished: FAILURE
```

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on jenkins-agent-01 in /home/jenkins/workspace/No-DSL-Method-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Parse JSON)
[Pipeline] script
[Pipeline] {
[Pipeline] readJSON
[Pipeline] echo
Parsed: value
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```




```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on jenkins-agent-01 in /home/jenkins/workspace/No-DSL-Method-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Parse JSON)
[Pipeline] script
[Pipeline] {
[Pipeline] readJSON
[Pipeline] echo
Parsed: value
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```


Conclusion

In this article, we've learned how to fix the *"No such DSL method"* error in Jenkins Pipeline. First, we looked at the causes of the error. After that, we provided possible solutions to it.

In short, we explored both the causes and solutions to the *"No such DSL method"* error in the Jenkins pipeline.

Task 25: Trigger Another Job from a Jenkins Pipeline

1. Objective

To trigger a downstream (child) Jenkins job from within a parent pipeline job using the built-in build step, including waiting for its completion and passing parameters.

2. Purpose

Real-world CI/CD workflows rarely consist of a single job. Modern automation requires orchestrating multiple jobs, where one job triggers another when its own stage completes. This "parent-child" pattern is essential for:

- **Pipeline chaining** — build job triggers test job triggers deploy job
- **Cross-team handoffs** — backend team's job triggers frontend team's job
- **Environment promotion** — dev deploy job triggers staging deploy job
- **Cross-repo automation** — changes in one repo trigger builds in another
- **Scheduled batches** — a master job triggers many sub-jobs
- **Conditional downstream** — only trigger if certain conditions are met
- **Parallel downstream** — trigger multiple jobs at once

4. Plugins Needed

Feature	Plugin Required
build step	Pipeline: Build Step plugin (usually pre-installed)
Parameterized triggering	Pipeline: Build Step (supports parameters)
Freestyle child job	None — built into Jenkins core
echo step	None — built into Pipeline

Note: The build step is provided by the Pipeline: Build Step plugin, which is typically installed by default with Jenkins. Verify under Manage Jenkins → Plugins → Installed.

5. Concepts Reference

5.1 Parent Job vs Child Job

Term	Meaning
Parent Job	The job that triggers other jobs
Child Job	The job being triggered
Downstream Job	Same as child — runs after the parent
Upstream Job	Same as parent — runs before the child

5.2 The build Step

The build step triggers another Jenkins job:

groovy
build job: "childJob"

Options:

Option	Meaning
job:	Name of the job to trigger (required)
wait:	true (default) waits for completion; false returns immediately
parameters:	List of parameters to pass
propagate:	true (default) fails parent if child fails
quietPeriod:	Seconds to wait before triggering

5.3 Synchronous vs Asynchronous

Mode	Syntax	Behavior
Synchronous	build job: 'childJob', wait: true	Parent waits for child to finish
Asynchronous	build job: 'childJob', wait: false	Parent continues immediately

5.4 Failure Propagation

By default, if the child job fails, the parent job also fails. To change this, set propagate: false — the child still runs, but its failure doesn't affect the parent

6. Step-by-Step Procedures

Step 0: Create the Child Job

Step 0.1: Click **New Item**

Step 0.2: Enter name: childJob

Step 0.3: Select **Freestyle project** → Click **OK**

Step 0.4: Scroll to **Build Steps** → Click **Add build step** → Select **Execute shell**

Step 0.5: Enter:

bash

```
echo "childJob executed at $(date)"
echo "This is the child job running"
```

Step 0.6: Click **Save**

Step 0.7: Click **Build Now**

Step 0B: Create the Parent Pipeline Job

Step 0B.1: Click **New Item**

Step 0B.2: Enter name: parentJob

Step 0B.3: Select **Pipeline** → Click **OK**

Step 0B.4: Leave the Pipeline script empty for now — we'll paste it in each method below.

Method 1: Basic Trigger (Synchronous, Default Behavior)

```
pipeline {
    agent any
    stages {
        stage('build') {
            steps {
                echo "parentJob is building..."
            }
        }
        stage('triggerChildJob') {
            steps {
                build job: "childJob", wait: true
            }
        }
    }
}
```

Why it works:

- build job: "childJob" finds the child job by name and schedules it
- wait: true (default) tells the parent to block until the child completes
- The child's console output is nested inside the parent's console output (indented)
- If the child fails, the parent fails (default propagate: true)

Method 2: Asynchronous Trigger (Fire and Forget)

```
pipeline {
  agent any
  stages {
    stage('build') {
      steps {
        echo "parentJob building..."
      }
    }
    stage('triggerChildAsync') {
      steps {
        build job: "childJob", wait: false
        echo "Child triggered — parent continuing without waiting."
      }
    }
    stage('continueParentWork') {
      steps {
        echo "Parent is doing other work in parallel with child..."
        sleep 5
        echo "Parent work done."
      }
    }
  }
}
```

Why it works:

- wait: false tells the parent to schedule the child and continue immediately
- The parent doesn't wait for the child's output
- The child runs concurrently on a separate executor
- The parent may finish before the child

]Method 3: Passing Parameters to the Child Job

Step 3.1: Update the Child Job to Accept Parameters

1. Go to childJob → **Configure**
2. Check **This project is parameterized**
3. Click **Add Parameter** → **String Parameter**
4. Name: MESSAGE
5. Default Value: default message
6. Click **Save**
7. Update the Build Step (Execute shell) to use the parameter:

bash

```
echo "Child received message: $MESSAGE"
```

Step 3.2: Update the Parent Pipeline

```
pipeline {
  agent any
  stages {
    stage('build') {
      steps {
        echo "parentJob building..."
      }
    }
    stage('triggerWithParameters') {
      steps {
        build(
          job: "childJob",
          wait: true,
          parameters: [
            string(name: 'MESSAGE', value: "Hello from parent at build ${env.BUILD_NUMBER}")
          ]
        )
      }
    }
  }
}
```

Why it works:

- The parameters: list passes named values to the child job
- The child job sees them as normal parameters ($MESSAGE in the shell)
- env.BUILD_NUMBER in the parent is evaluated before triggering, so the child receives the parent's build number as a string

Method 4: Continue Parent Even If Child Fails

```
pipeline {
  agent any
  stages {
    stage('build') {
      steps {
        echo "parentJob building..."
      }
```

```
    }
    stage('triggerWithPropagateFalse') {
      steps {
        script {
          def result = build(
            job: "childJob",
            wait: true,
            propagate: false,
            parameters: [
              string(name: 'MESSAGE', value: "test message")
            ]
          )
          echo "Child result: ${result.result}"
          echo "Child build number: ${result.number}"
        }
      }
    }
    stage('parentContinuesAnyway') {
      steps {
        echo "Parent continues regardless of child outcome."
      }
    }
  }
}
```

Why it works:

- propagate: false tells Jenkins not to fail the parent even if the child fails
- The build step returns a RunWrapper object with useful properties:
  - .result — SUCCESS, FAILURE, ABORTED, UNSTABLE
  - .number — the child's build number
  - .absoluteUrl — the URL of the child build
  - .description — the build description

When to use: Triggering non-critical jobs (reporting, notifications, optional tests) where the parent should continue regardless.

Method 5: Trigger Multiple Child Jobs in Parallel

```
pipeline {
  agent any
  stages {
    stage('build') {
      steps {
        echo "parentJob building..."
      }
    }
    stage('triggerMultipleChildren') {
      steps {
        script {
```

```
                def children = ['childJob', 'childJob2', 'childJob3']
                def branches = [:]

                children.each { jobName ->
                    branches["Trigger ${jobName}"] = {
                        build(
                            job: jobName,
                            wait: true,
                            propagate: false
                        )
                    }
                }

                parallel branches
            }
        }
    }
}
```

Method 6: Conditional Job Triggering

```
pipeline {
    agent any
    parameters {
        booleanParam(name: 'RUN_INTEGRATION', defaultValue: false, description: 'Run integration
tests?')
    }
    stages {
        stage('build') {
            steps {
                echo "parentJob building..."
            }
        }
        stage('triggerOnCondition') {
            when {
                expression { params.RUN_INTEGRATION }
            }
            steps {
                build(
                    job: "childJob",
                    wait: true,
                    parameters: [
                        string(name: 'MESSAGE', value: "Running integration tests")
                    ]
                )
            }
        }
        stage('alwaysRuns') {
```

```
        steps {
            echo "This stage always runs."
        }
    }
  }
}
```

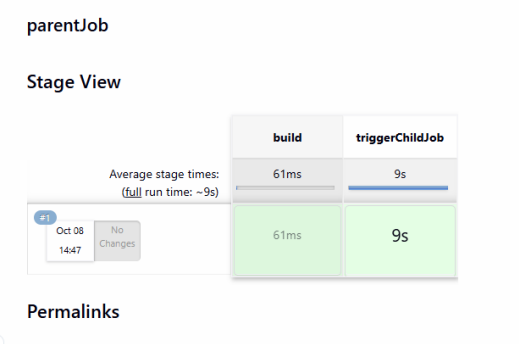
Step 6.2: Save → Build with Parameters → Run twice: once with RUN_INTEGRATION unchecked, once checked.

Unchecked output:

text

parentJob building...
Stage "triggerOnCondition" skipped due to when conditional
This stage always runs.
Finished: SUCCESS

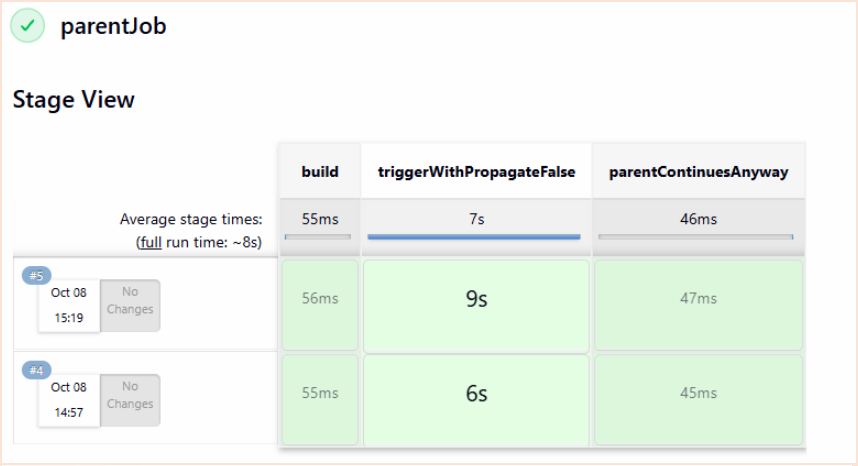
Implementation:





```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/parentJob
[Pipeline] {
[Pipeline] stage
[Pipeline] { (build)
[Pipeline] echo
parentJob building...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (triggerChildAsync)
[Pipeline] build (Scheduling childJob)
Scheduling project: childJob
[Pipeline] echo
Child triggered — parent continuing without waiting.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (continueParentWork)
[Pipeline] echo
Parent is doing other work in parallel with child...
[Pipeline] sleep
Sleeping for 5 sec
[Pipeline] echo
Parent work done.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
```

```
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/parentJob
[Pipeline] {
[Pipeline] stage
[Pipeline] { (build)
[Pipeline] echo
parentJob building...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (triggerWithPropagateFalse)
[Pipeline] script
[Pipeline] {
[Pipeline] build (Building childJob)
Scheduling project: childJob
Starting building: childJob #4
Build childJob #4 completed: SUCCESS
[Pipeline] echo
Child result: SUCCESS
[Pipeline] echo
Child build number: 4
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (parentContinuesAnyway)
[Pipeline] echo
Parent continues regardless of child outcome.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```




```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/parentJob
[Pipeline] {
[Pipeline] stage
[Pipeline] { (build)
[Pipeline] echo
parentJob building...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (triggerWithPropagateFalse)
[Pipeline] script
[Pipeline] {
[Pipeline] build (Building childJob)
Scheduling project: childJob
Starting building: childJob #5
Build childJob #5 completed: SUCCESS
[Pipeline] echo
Child result: SUCCESS
[Pipeline] echo
Child build number: 5
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (parentContinuesAnyway)
[Pipeline] echo
Parent continues regardless of child outcome.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
```

Conclusion

In this article, we demonstrated how to create a job that internally triggers another job in Jenkins. As a first step, we created a sample freestyle Jenkins job and then created a pipeline job to internally trigger it.

Task 26: How to Inject Git Secrets in Jenkins

This is not "Jenkins storing a Git password."

The actual secret is encrypted with **GPG**, the encrypted file is committed to Git, and Jenkins temporarily receives the **GPG private key + ownertrust + passphrase** so the build can decrypt the file when needed.

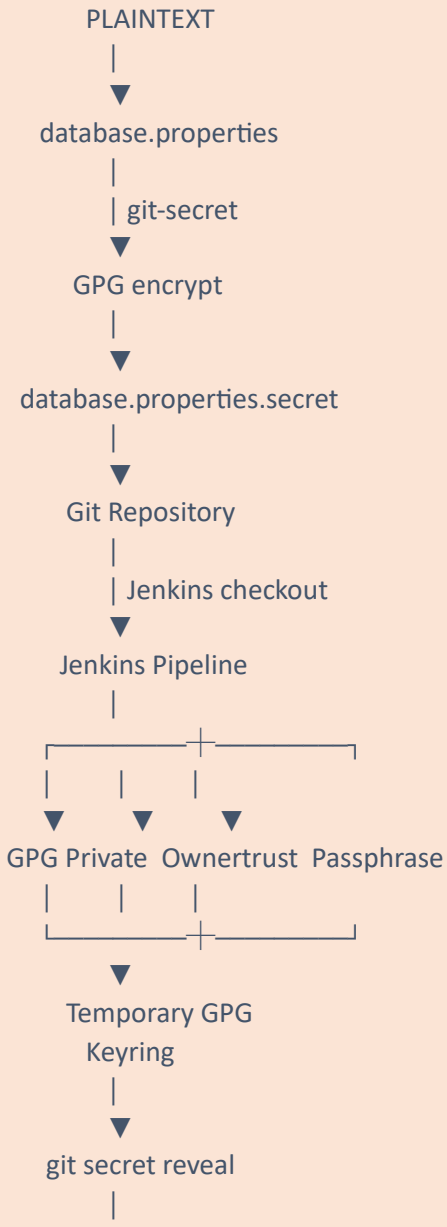
1. What are we actually trying to solve?

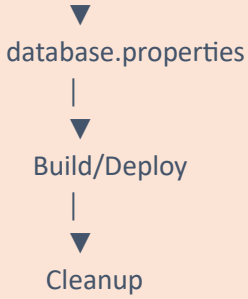
Imagine our application needs this:

```
DB_USERNAME=devopsuser
DB_PASSWORD=FreshLabPassword@2026
DB_HOST=localhost
DB_PORT=3306
```

The **wrong approach** is:
database.properties → Git repository → Jenkins

The better approach is:





2. Understand the four technologies first

This is the most important part to understand before running commands.

Git

Git stores our application source code and, in this design, the **encrypted secret file**.

Git should contain: database.properties.secret

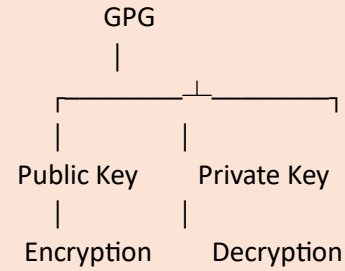
Git should **not** contain: database.properties

The source explicitly verifies this using git ls-files.

What is GPG?

GPG provides the cryptographic keys.

We create:



Conceptually:



Then:

4. What is git-secret?

git-secret is the layer that connects: Git + GPG
Instead of manually running GPG encryption for every secret file, we can do:
→ git secret add database.properties
→ git secret hide
and get: database.properties.secret

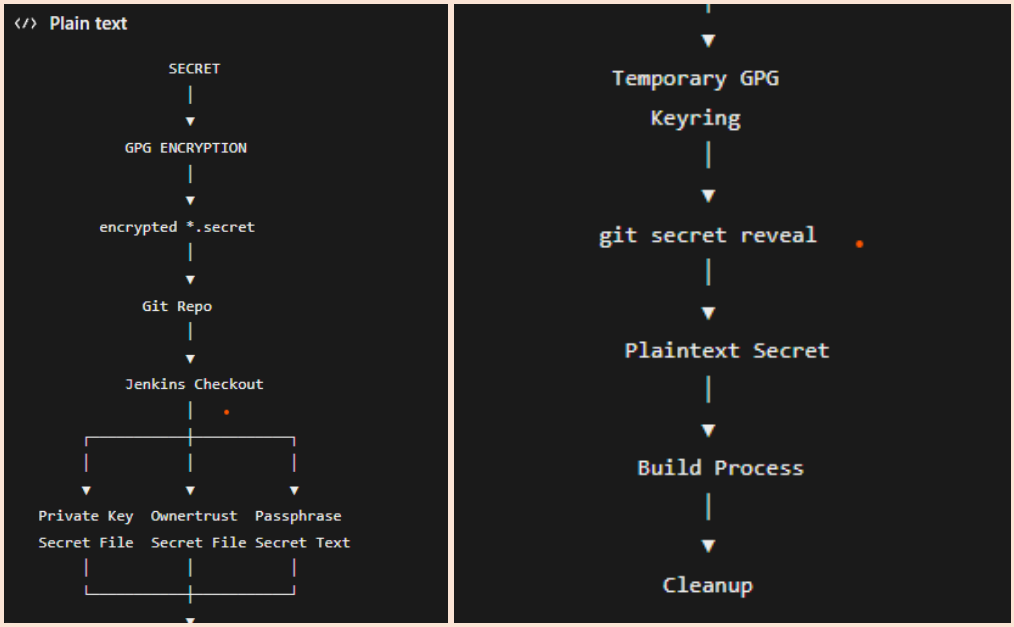
5. What is Jenkins Credentials?

This is another critical concept.
Jenkins does **not** need the actual password inside the Jenkinsfile.
Instead, Jenkins stores credentials separately.
For this lab we have **three Jenkins credentials**:

Jenkins Credential	Type	Purpose
fresh-gpg-private-key	Secret File	GPG private key
fresh-gpg-ownertrust	Secret File	GPG trust information
fresh-gpg-passphrase	Secret Text	Password protecting the GPG key

It does **not** contain: FreshLabPassword@2026

6. The complete security model



7. Why are there THREE Jenkins credentials?

Credential 1 — Private Key
fresh-gpg-private-key → Contains: -----BEGIN PGP PRIVATE KEY BLOCK----- ...

This allows GPG to decrypt the encrypted secret.

Credential 2 — Ownertrust

fresh-gpg-ownertrust → This contains GPG trust information.
→ gpg --import-ownertrust "$GPG_OWNERTRUST"

Credential 3 — Passphrase → This is the passphrase that protects the private GPG key.

▼

fresh-gpg-passphrase

8. Why can't we just put the GPG private key in Git?

Absolutely not.

Git Repository
 |
 ├— application code
 ├— encrypted secret
 └— GPG private key ❌

That destroys the security model.
Instead:
Git Repository
 |
 ├—— application code
 └—— encrypted secret ✅

Jenkins Credentials
 |
 ├—— GPG private key ✅
 ├—— GPG ownertrust ✅
 └—— GPG passphrase ✅

9. Why use a temporary GPG home?

export GNUPGHOME="$WORKSPACE/.gnupg"

Instead of using some existing GPG configuration on the EC2 server, Jenkins creates:
Workspace
 |
 └—— .gnupg/
  |
  ├—— imported private key
  └—— imported trust

Then:
git secret reveal
uses that temporary GPG environment.
After the build:
rm -rf "$WORKSPACE/.gnupg"
So the workflow becomes:
Create temporary GPG environment
↓
Import credentials
↓
Decrypt
↓
Use secret
↓
Delete GPG environment

10. Why withCredentials()?
This is the Jenkins mechanism that temporarily exposes credentials to a build.

```
withCredentials([
   string(
      credentialsId: 'fresh-gpg-passphrase',
      variable: 'GPG_PASSPHRASE'
   )
]) {
   ...
}
```

Jenkins makes the credential available to the shell as: GPG_PASSPHRASE

Run: sudo dnf update -y
sudo dnf –version

Install Required Packages
sudo dnf install -y git gnupg2 gawk make file
git –version
gpg –version
make –version
gawk –version

Install git-secret
The official git-secret documentation provides a manual installation method:
cd /opt
Clone git-secret: sudo git clone https://github.com/sobolevn/git-secret.git
cd /opt/git-secret
Build it: sudo make build
Install it: sudo PREFIX="/usr/local" make install

Now verify: git secret –version
Also try: git-secret –version
Verify Jenkins User: id Jenkins
getent passwd Jenkins
Now verify that Jenkins can execute the commands:
→ sudo -u jenkins git –version
→ sudo -u jenkins gpg –version
→sudo -u jenkins git-secret –version
If git-secret is available through /usr/local/bin, Jenkins should be able to find it.
→sudo -u jenkins which git
→sudo -u jenkins which gpg
→sudo -u jenkins which git-secret

Create a Fresh Lab Directory
→ sudo mkdir -p /opt/jenkins-git-secret-lab
→ sudo chown -R jenkins:jenkins /opt/jenkins-git-secret-lab
→ ls -ld /opt/jenkins-git-secret-lab

Switch to Jenkins User
sudo -iu Jenkins
Create a Fresh Git Repository
→ cd /opt/jenkins-git-secret-lab
→ mkdir git-secret-demo
→ cd git-secret-demo
→ git init

Configure Git identity for this lab:
git config user.name "Jenkins Git Secret Lab"
git config user.email jenkins-git-secret@example.com
git config –list

Create the GPG Key
git-secret uses GPG public/private keys. The public key is used for encryption and the private key is required for decryption
gpg --full-generate-key
You will see a menu similar to: Please select what kind of key you want: 1
Then: What keysize do you want? → Enter: → 4096 →For expiration: →0
This means the key does not expire for this lab.

Verify the GPG Key
gpg --list-secret-keys --keyid-format LONG
Test the GPG Key
Create a temporary file: → echo "GPG test message" > gpg-test.txt
Encrypt it: → gpg --encrypt --recipient jenkins-git-secret@example.com gpg-test.txt
ls -l → Decrypt: → gpg --decrypt gpg-test.txt.gpg
Now remove the test: rm -f gpg-test.txt gpg-test.txt.gpg

Initialize git-secret: Make sure you are inside: /opt/jenkins-git-secret-lab/git-secret-demo
Now: git secret init
The official git-secret initialization creates .gitsecret/ and configures Git handling for encrypted .secret files

ls -la → You should see:
.git
.gitignore
.gitsecret

Check:
find .gitsecret -maxdepth 3 -type f -print
Tell git-secret Which GPG User Can Access Secrets
Run: → git secret tell jenkins-git-secret@example.com
Verify: → git secret whoknows
Create the Plaintext Secret
cat > database.properties <<'EOF'
DB_USERNAME=devopsuser
DB_PASSWORD=FreshLabPassword@2026
DB_HOST=localhost
DB_PORT=3306
EOF

Check it: cat database.properties
Add the Secret to git-secret
Run: git secret add database.properties
Check: git secret list
Encrypt the Secret
Run: git secret hide
Now check: ls -la
Verify the Encrypted File
Run: file database.properties.secret
Try: cat database.properties.secret
Remove the Plaintext Secret
rm -f database.properties
ls -la
git status
==========
Method 1: Generate GPG Keys
Method 2: Initialize git-secret in a Repository
Method 3: Store and Encrypt a Secret
Method 4: Export GPG Key for Jenkins
Method 5: Create Jenkins Credentials
Step 5.1: Navigate to Credentials

1. Go to **Manage Jenkins → Credentials**
2. Click **System** (under "Stores scoped to Jenkins")
3. Click **Global credentials (unrestricted)**
4. Click **+ Add Credentials**

Step 5.2: Create GPG Secret Key Credential (Secret file)

Fill in:

Field	Value
Kind	Secret file
Scope	Global
File	Upload gpg-secret.key
ID	gpg-secret
Description	GPG Secret Key

Click Create.

Step 5.3: Create GPG Trust Store Credential (Secret file)

Click + Add Credentials again:

Field	Value
Kind	Secret file
Scope	Global
File	Upload gpg-ownertrust.txt
ID	gpg-ownertrust
Description	GPG Owner Trust

Click Create.

Step 5.4: Create GPG Passphrase Credential (Secret text)

Click + Add Credentials again:

Field	Value
Kind	Secret text
Scope	Global
Secret	MySecretPassphrase (the passphrase from Step 1.1)
ID	gpg-passphrase
Description	GPG Passphrase

Click Create.

Step 5.5: Verify

Three credentials now appear in the Global credentials list:

gpg-secret (Secret file)

gpg-ownertrust (Secret file)

gpg-passphrase (Secret text)

Method 6: Use Git Secrets in a Jenkins Pipeline

Step 6.1: Ensure git-secret is on the Agent

Verify git-secret is installed on the Jenkins node:

which git-secret

If running on the Built-in Node, install it on the Jenkins host (Step 0).

Step 6.2: Create the Pipeline Job

1. Click **New Item** → Name: Git-Secrets-Demo → Select **Pipeline** → **OK**

Step 6.3: Paste the Pipeline Script

```
pipeline {
    agent any
    environment {
        gpg_secret     = credentials('gpg-secret')
        gpg_trust      = credentials('gpg-ownertrust')
        gpg_passphrase = credentials('gpg-passphrase')
    }
    stages {
        stage('Import GPG Keys') {
            steps {
                sh '''
                    gpg --batch --import $gpg_secret
                    gpg --import-ownertrust $gpg_trust
                    gpg --list-secret-keys
                '''
            }
        }
        stage('Reveal Git Secret') {
            steps {
                sh '''
                    cd $WORKSPACE
                    git secret reveal -p "$gpg_passphrase"
                    echo "Decrypted password:"
                    git secret cat dbpassword.txt
                '''
            }
        }
    }
}
```

Step 6.4: Save → Build Now

Method 7: Use Git Secrets in a Freestyle Job

Step 7.1: Create a Freestyle Job

1. Click **New Item** → Name: Git-Secrets-Freestyle → Select **Freestyle project** → **OK**

Step 7.2: Configure Build Environment

1. Scroll to **Build Environment**
2. Check **Use secret text(s) or file(s)**
3. Click **Add** three times:
    - Binding 1: **Secret file** → Credentials: gpg-secret → Variable: GPG_SECRET
    - Binding 2: **Secret file** → Credentials: gpg-ownertrust → Variable: GPG_TRUST
    - Binding 3: **Secret text** → Credentials: gpg-passphrase → Variable: GPG_PASSPHRASE

Step 7.3: Add Build Steps

1. Scroll to **Build Steps** → **Add build step** → **Execute shell**
2. Enter:

```
gpg --batch --import $GPG_SECRET
gpg --import-ownertrust $GPG_TRUST
cd $WORKSPACE
git secret reveal -p "$GPG_PASSPHRASE"
git secret cat dbpassword.txt
```

Step 7.4: Save → Build Now

Method 8: Verify Secret Masking

Step 8.1: Add a diagnostic step to the pipeline:

```
stage('Verify Masking') {
   steps {
      sh '''
         echo "Passphrase variable holds: $gpg_passphrase"
         echo "Secret key path: $gpg_secret"
      '''
   }
}
```

Conclusion

In this tutorial, we have seen how to use git secrets with both Jenkins pipelines and traditional jobs. This is an easy way to provide access to sensitive data to your CI/CD pipelines.

Task 27: Scripted vs Declarative Pipelines in Jenkins

1. Objective

To understand the differences between **Declarative** and **Scripted** Jenkins pipelines, compare their syntax, structure, and capabilities, and determine when to use each approach.

2. Purpose

Jenkins provides two ways to write pipelines, both using the same underlying execution engine but with fundamentally different syntax and philosophy:

- **Declarative Pipelines** — structured, opinionated, easier to read and maintain
- **Scripted Pipelines** — flexible, Groovy-based, more powerful but harder to learn

5. Concepts Reference

5.1 Core Structural Differences

Feature	Declarative	Scripted
Top-level block	pipeline { }	node { }
Agent declaration	agent { } directive	node('label') { }
Stages	stages { stage('X') { } }	stage('X') { } directly
Steps wrapper	Required: steps { }	Not needed
Groovy code	Wrapped in script { }	Anywhere in the block
Directives	environment, options, parameters, tools, when, post, etc.	Not available — use Groovy instead
Learning curve	Easier	Steeper (requires Groovy)
Validation	Syntax validated at parse time	Errors often at runtime

5.2 Declarative Pipeline

A structured, opinionated syntax using a pipeline { } block. The approach is recent and designed for readability.

Main sections:

Section	Purpose
pipeline	Contains the whole pipeline
agent	Defines the machine that executes the pipeline
stages	Declares the stages of the pipeline
steps	Small operations inside a stage

Additional directives:

- tools — declare build tool versions
- triggers — set build triggers
- environment — set environment variables
- parameters — declare build parameters
- post — actions after pipeline completes
- when — conditional stage execution
- input — prompt users for input

5.3 Scripted Pipeline

A Groovy-based DSL with fewer restrictions. This was the first version of pipeline-as-code.

Main blocks:

Block	Purpose
node	Specifies the machine that executes the pipeline
stage	Groups steps into a separate operation

Key trait: Steps contain all logic — no directives. Complex logic is implemented directly in Groovy.

5.4 The script Directive

The script directive lets a Declarative pipeline run arbitrary Groovy — effectively embedding Scripted logic inside a Declarative structure.

6. Step-by-Step Procedures

Step 0: Create the Jobs

Create two pipeline jobs to compare:

1. **New Item** → Name: Declarative-Demo → **Pipeline** → **OK**
2. **New Item** → Name: Scripted-Demo → **Pipeline** → **OK**

Method 1: Hello World in Declarative Syntax

```
pipeline {
  agent any
  stages {
    stage('Hello World') {
      steps {
        sh 'echo Hello World'
      }
    }
  }
}
```

Structure:

- pipeline { } — root block
- agent any — run on any available agent
- stages { } — declare all stages
- stage('...') { } — one stage
- steps { } — the operations inside the stage
- sh '...' — a step that runs a shell command

Method 2: Hello World in Scripted Syntax

```
node {
   stage('Hello World') {
      sh 'echo Hello World'
   }
}
```

Structure:

- node { } — root block (replaces both pipeline and agent)
- stage('...') { } — one stage, containing its steps directly
- sh '...' — a step

Method 3: Three-Stage Pipeline — Declarative

```
pipeline {
   agent any
   tools {
      maven 'maven'
   }
   stages {
      stage('Test') {
         steps {
            git 'https://github.com/user/project.git'
            sh 'mvn test'
            archiveArtifacts artifacts: 'target/surefire-reports/**'
         }
      }
      stage('Build') {
         steps {
            sh 'mvn clean package -DskipTests'
            archiveArtifacts artifacts: 'target/*.jar'
         }
      }
      stage('Deploy') {
         steps {
            sh 'echo Deploy'
         }
      }
   }
}
```

Method 4: Three-Stage Pipeline — Scripted

```
node {
   stage('Test') {
      git 'https://github.com/user/project.git'
      sh 'mvn test'
      archiveArtifacts artifacts: 'target/surefire-reports/**'
   }
   stage('Build') {
```

```
        sh 'mvn clean package -DskipTests'
        archiveArtifacts artifacts: 'target/*.jar'
    }
    stage('Deploy') {
        sh 'echo Deploy'
    }
}
```

Implementation:

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on jenkins-agent-01 in /home/jenkins/workspace/Declarative-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Hello World)
[Pipeline] sh
+ echo Hello World
Hello World
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on jenkins-agent-01 in /home/jenkins/workspace/Scripted-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Hello World)
[Pipeline] sh
+ echo Hello World
Hello World
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Conclusion

→Scripted and declarative pipelines follow the same goal and use the same pipeline sub-system under the hood. The major differences between them are flexibility and syntax. **They're just two different tools for solving the same problem, thus, we can and should use them interchangeably.**

→The succinct syntax of declarative pipelines will ensure a faster and smoother entrance to this field. At the same time, scripted pipelines may provide more power to more experienced users. **In order to get the best from both worlds, we can leverage declarative pipelines with script directives.**

Task 28: Intro to Jenkins 2 and the Power of Pipelines

1. Objective

To understand the Jenkins 2 Pipeline feature, set up a complete continuous delivery pipeline (Scripted syntax) that clones a project, compiles it in parallel with static analysis, runs unit and integration tests in parallel, and deploys the application — and to review the resulting analysis reports in the Jenkins UI.

2. Purpose

Jenkins 2 introduced **Pipelines** — a first-class feature that lets teams define the entire CI/CD process as code. Unlike traditional Freestyle jobs (where build steps are configured through the UI), Pipelines are:

- **Versioned** — stored in a Jenkinsfile inside the repository
- **Persistent** — survive Jenkins restarts
- **Pausable** — can wait for manual approval
- **Parallelizable** — run independent actions concurrently
- **Extensible** — integrate with plugins and other systems

The example pipeline in this task demonstrates a realistic continuous delivery flow:

text

Clone → Compilation ‖ Static Analysis → Unit Tests ‖ Integration Tests → Deploy

3. Concepts Reference

3.1 What Is Jenkins 2?

Jenkins 2 (released 2016) added Pipelines as a first-class feature, along with improvements to security, configuration-as-code, and plugin management. Modern Jenkins (2.580.1 in your setup) inherits all of these.

3.2 What Is a Pipeline?

A Pipeline is a way to define Jenkins steps as code using a Domain-Specific Language (DSL). Two syntaxes exist:

Syntax	Style
Declarative	Structured, pipeline { } block
Scripted	Imperative Groovy, node { } block

4. Step-by-Step Procedures

Step 0: Install Required Plugins

1. Go to **Manage Jenkins → Plugins → Available**
2. Search and install:
    - **Checkstyle Plugin**
    - **Static Analysis Collector Plugin**
    - **JUnit Plugin** (usually pre-installed)
    - **Pipeline: Stage View** (usually pre-installed)
    - **Git Plugin** (usually pre-installed)
3. Restart Jenkins if prompted

Step 0B: Verify the Pipeline Plugin

1. Go to **Manage Jenkins → Plugins → Installed**

2. Search Pipeline
3. Confirm **Pipeline** and **Pipeline: Stage View** are installed and enabled

Step 0C: Create the Pipeline Job

1. Click **New Item** → Name: Jenkins2-Pipeline-Demo → Select **Pipeline** → **OK**
2. Scroll to the **Pipeline** section
3. Select **Pipeline script** as the definition
4. Check **Use Groovy Sandbox**
5. Keep the script box ready for the script in Method 1

Method 1: The Full Continuous Delivery Pipeline (Scripted)

```
node {
   stage('Clone the project') {
      git 'https://github.com/eugenp/tutorials.git'
   }

   dir('spring-jenkins-pipeline') {
      stage('Compilation and Analysis') {
         parallel(
            'Compilation': {
               sh './mvnw clean install -DskipTests'
            },
            'Static Analysis': {
               stage('Checkstyle') {
                  sh './mvnw checkstyle:checkstyle'
                  step([$class: 'CheckStylePublisher',
                        canRunOnFailed: true,
                        defaultEncoding: '',
                        healthy: '100',
                        pattern: '**/target/checkstyle-result.xml',
                        unHealthy: '90',
                        useStableBuildAsReference: true
                  ])
               }
            }
         )
      }

      stage('Tests and Deployment') {
         parallel(
            'Unit tests': {
               stage('Running unit tests') {
                  try {
                     sh './mvnw test -Punit'
                  } catch (err) {
                     step([$class: 'JUnitResultArchiver',
                           testResults: '**/target/surefire-reports/TEST-*UnitTest.xml'])
                     throw err
                  }
```

```
                step([$class: 'JUnitResultArchiver',
                    testResults: '**/target/surefire-reports/TEST-*UnitTest.xml'])
                }
            },
            'Integration tests': {
                stage('Running integration tests') {
                    try {
                        sh './mvnw test -Pintegration'
                    } catch (err) {
                        step([$class: 'JUnitResultArchiver',
                            testResults: '**/target/surefire-reports/TEST-*IntegrationTest.xml'])
                        throw err
                    }
                    step([$class: 'JUnitResultArchiver',
                        testResults: '**/target/surefire-reports/TEST-*IntegrationTest.xml'])
                }
            }
        )

        stage('Staging') {
            sh 'pid=$(lsof -i:8989 -t); kill -TERM $pid || kill -KILL $pid'
            withEnv(['JENKINS_NODE_COOKIE=dontkill']) {
                sh 'nohup ./mvnw spring-boot:run -Dserver.port=8989 &'
            }
        }
    }
  }
}
```

Method 3: Verify Parallelism in the Stage View

Step 3.1: Go to the job's main page

Step 3.2: Look for **Stage View** (usually below Build History)

Step 3.3: Confirm:

- **Compilation** and **Static Analysis** appear **side by side** (parallel)
- **Unit tests** and **Integration tests** appear **side by side** (parallel)
- **Staging** appears after both test stages (sequential)

Step 3.4: Hover over a stage cell → click **Logs** to see the log messages for that stage.

Step 3.5: Compare durations:

- Sequential pipeline: Compilation + Static Analysis + Unit tests + Integration tests (sum)
- Parallel pipeline: max(Compilation, Static Analysis) + max(Unit tests, Integration tests) — much faster

Method 4: Review the Checkstyle Report
Step 4.1: Go to the build page
Step 4.2: Click **Checkstyle Warnings** (left sidebar)
Step 4.3: Observe:

- Total number of warnings
- Warnings grouped by priority (High / Normal / Low)
- The trend graph (if multiple builds)
- A table of files with warnings

Step 4.4: Click the **Details** tab to see:

- Each warning's line number
- The warning message
- The source rule (e.g., JavadocMethod)

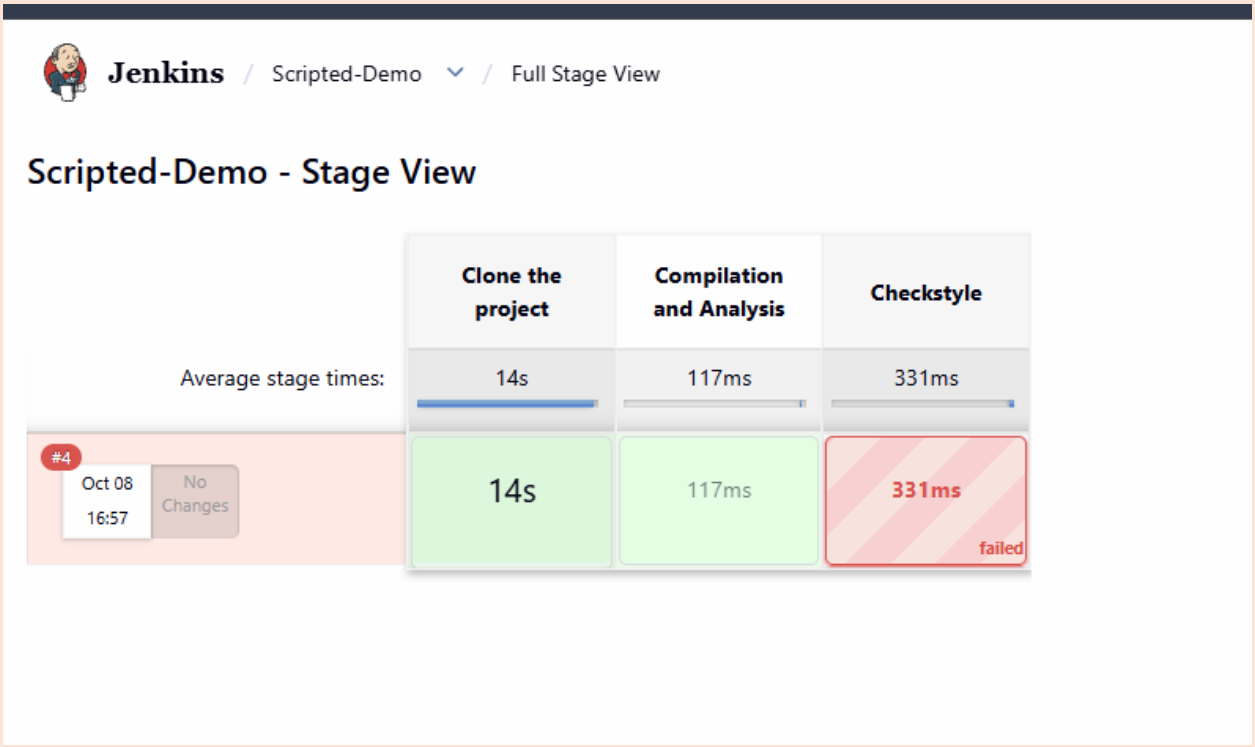
Method 5: Review the JUnit Test Results
Step 5.1: Go to the build page
Step 5.2: Click **Test Result** (left sidebar)
Step 5.3: Observe:

- Total tests run
- Failures, errors, skipped
- Duration
- Per-package breakdown
- Per-test details (name, duration, status)

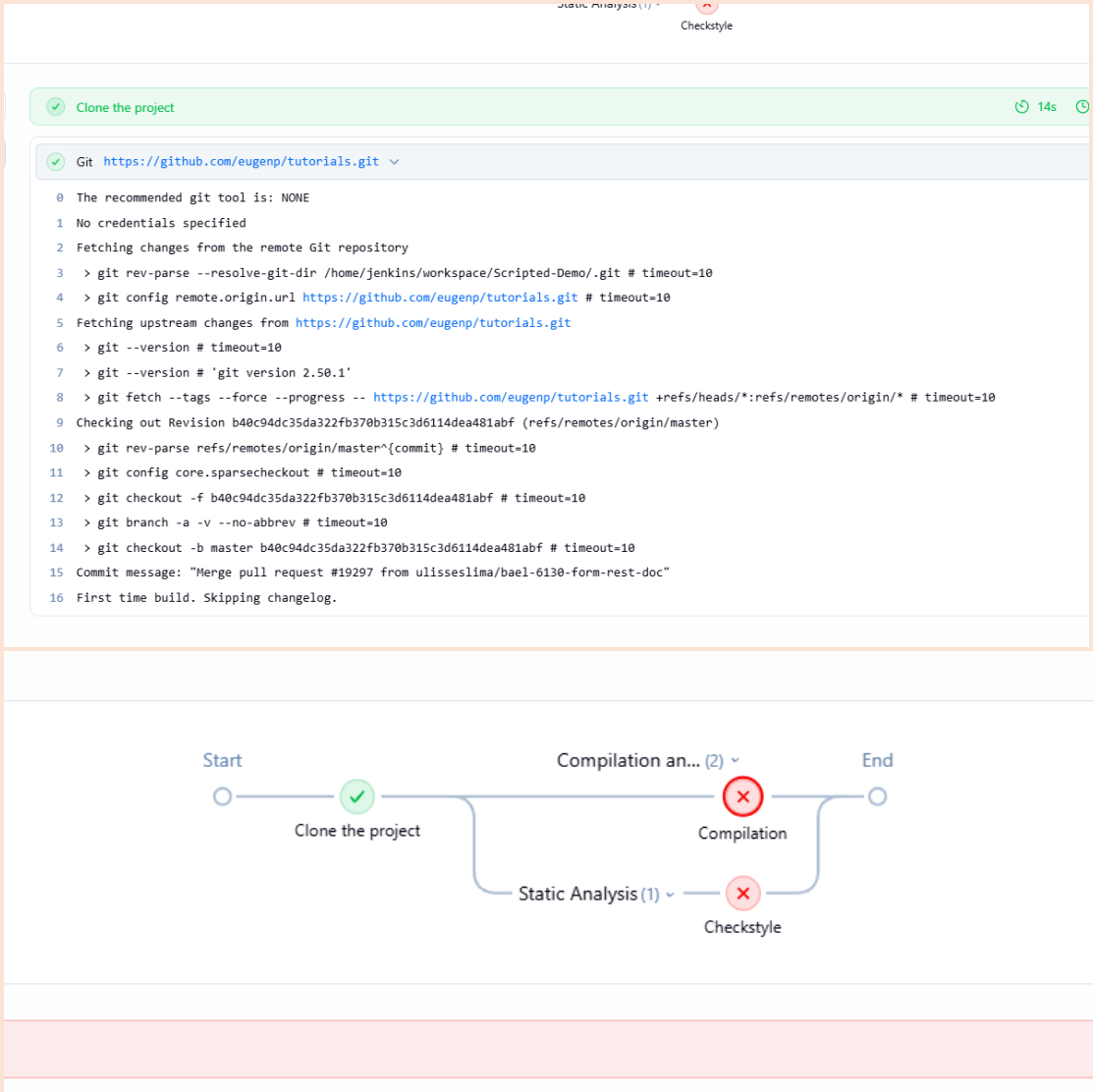
Step 5.4: Click on a package name (e.g., com.baeldung) to see individual tests.
Why this matters: Jenkins provides a single interface for test results from any test framework that produces JUnit-format XML. This is the standard for CI test reporting.

Method 6: Understand Failure Behavior
Step 6.1: Modify the pipeline to simulate a test failure:
In the Unit tests stage, replace the sh block with one that generates a failing test report:

Screencshots:




```
[Pipeline] { (Branch: Compilation)
[Pipeline] { (Branch: Static Analysis)
[Pipeline] sh
[Pipeline] stage
[Pipeline] { (Checkstyle)
[Pipeline] sh
[Compilation] + ./mvnw clean install -DskipTests
/home/jenkins/workspace/Scripted-Demo/spring-jenkins-pipeline@tmp/durable-b0003372/script.sh.copy: line 1: ./mvnw: No such file or directo
[Pipeline] }
Failed in branch Compilation
[Static Analysis] + ./mvnw checkstyle:checkstyle
/home/jenkins/workspace/Scripted-Demo/spring-jenkins-pipeline@tmp/durable-49efd094/script.sh.copy: line 1: ./mvnw: No such file or directo
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
Failed in branch Static Analysis
[Pipeline] // parallel
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // dir
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
ERROR: script returned exit code 127
Finished: FAILURE
```



Conclusion

In this article, we set up a simple continuous delivery environment to run and show static code analysis and test report in Jenkins via a *Pipeline* job.

Task 29: Simple Jenkins Pipeline with Marathon and Mesos

1. Overview

This task walks through building a Continuous Delivery pipeline in Jenkins that:

1. Clones a Spring Boot web application from GitHub
2. Builds it with Maven into a deployable WAR file
3. Packages the WAR into a Docker image
4. Pushes the image to Docker Hub
5. (Optional) Deploys the image to a Mesos cluster via Marathon

The pipeline demonstrates how Jenkins orchestrates a real-world CD flow: source → build → containerize → publish → deploy.

2. Objective

To automate the build, containerization, and publishing of a Java web application using Jenkins, Maven, Docker, and Docker Hub — with optional Marathon/Mesos deployment.

3. Purpose

Traditional Java web app deployments involve manual WAR uploads, server configuration, and restart procedures. This pipeline automates the entire flow:

- **Consistency** — every build produces an identical, versioned Docker image
- **Traceability** — each image is tagged with the Jenkins build number
- **Reproducibility** — the same pipeline runs on every code change
- **Speed** — from commit to deployable container in minutes
- **Portability** — Docker images run anywhere Docker runs

4. Environment Used

Item	Value
Jenkins Version	2.580.1
Jenkins Host	Linux (Ubuntu/Debian)
Java	OpenJDK 17
Maven	3.8.x (installed on Jenkins host)
Docker	24.x or later (installed on Jenkins host)
Git	2.x
Docker Hub Account	sunith18
Repository	https://github.com/Sunith3210/sabear_simplecutomerapp.git
Application Type	Java WAR (Spring/JSP web app)

5. Plugins Required

Plugin	Purpose
Git Plugin	Clone the repository
EnvInject Plugin	Inject DOCKER_PASSWORD securely
Credentials Binding Plugin	Bind Jenkins credentials to environment variables
Marathon Plugin (optional)	Deploy to Marathon/Mesos

6. Architecture

text

GitHub Repo → Jenkins → Maven Build → WAR → Docker Image → Docker Hub

↓

(Optional) Marathon → Mesos

7. Repository Files

The GitHub repository contains these key files:

File	Purpose
pom.xml	Maven project definition; produces SimpleCustomerApp-<BUILD_NUMBER>-SNAPSHOT.war
Dockerfile	Instructions to build the Docker image
Dockerise.sh	Bash script that logs in to Docker Hub, builds, and pushes the image
Jenkinsfile	(Optional) Pipeline definition
src/	Java source code
WebContent/	Web resources

8. Step-by-Step Procedures

Step 1: Verify Prerequisites on the Jenkins Host

```
# Verify Maven
mvn -version
sudo -u jenkins mvn -version

# Verify Docker
docker --version
sudo -u jenkins docker ps
```

If any command fails, install the missing tool:

```
# Install Maven
sudo apt update && sudo apt install -y maven

# Install Docker
sudo apt update && sudo apt install -y docker.io
sudo usermod -aG docker jenkins
sudo systemctl restart Jenkins
```

Step 2: Verify the Dockerfile

he Dockerfile in the repo root must:

- Use a valid base image (e.g., tomcat:10-jdk17-temurin)
- Copy the WAR produced by Maven into Tomcat's webapps folder
- Use a **wildcard** for the WAR name, because the version changes on every build

Create in guthub repo → new file In repo

Correct Dockerfile**:**

```
FROM tomcat:10-jdk17-temurin
RUN rm -rf /usr/local/tomcat/webapps/*
COPY target/SimpleCustomerApp-*.war /usr/local/tomcat/webapps/ROOT.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

Key point: SimpleCustomerApp-*.war matches any build number. A hardcoded version like SimpleCustomerApp-2-SNAPSHOT.war will fail on the next build.

Step 3: Verify the Dockerise.sh

The Dockerise.sh script builds and pushes the image:

```
#!/usr/bin/env bash
set -e
echo "$DOCKER_PASSWORD" | docker login -u sunith18 --password-stdin
docker build -t sunith18/sabear_simplecutomerapp:$BUILD_NUMBER .
docker push sunith18/sabear_simplecutomerapp:$BUILD_NUMBER
docker logout
```

Notes:

- $DOCKER_PASSWORD is injected by Jenkins at runtime
- $BUILD_NUMBER is provided automatically by Jenkins
- --password-stdin avoids the "insecure" warning
- docker logout cleans up credentials after pushing

Step 4: Create the Jenkins Credential

1. Go to **Manage Jenkins → Credentials → System → Global credentials**
2. Click **+ Add Credentials**
3. Configure:
    - **Kind:** Secret text
    - **Scope:** Global
    - **Secret:** your Docker Hub access token (e.g., dckr_pat_...)
    - **ID:** docker-hub-token
    - **Description:** Docker Hub access token
4. Click **Create**

Step 5: Create the Jenkins Freestyle Job

1. **New Item** → Name: marathon-mesos-demo → Select **Freestyle project** → **OK**
2. **Source Code Management:**
    - Git
    - Repository URL: https://github.com/Sunith3210/sabear_simplecutomerapp.git
    - Branch: */master
3. **Build Environment:**
    - Check **Use secret text(s) or file(s)**
    - Add a binding:
        - **Variable:** DOCKER_PASSWORD
        - **Credentials:** docker-hub-token
4. **Build Step 1 — Execute shell:**

bash

```
mvn clean install
```

5. **Build Step 2 — Execute shell:**

bash

```
chmod +x Dockerise.sh
./Dockerise.sh
```

6. Click **Save**

Step 6: Run the Build

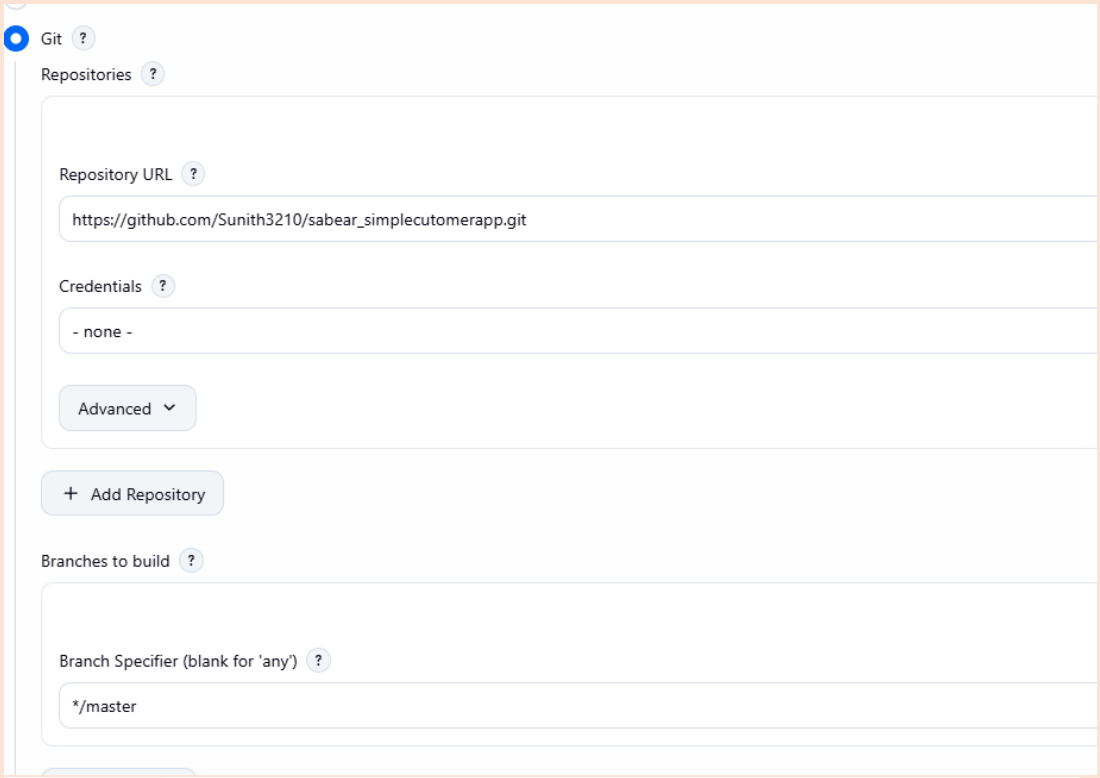
1. Click **Build Now**
2. Watch the **Console Output**

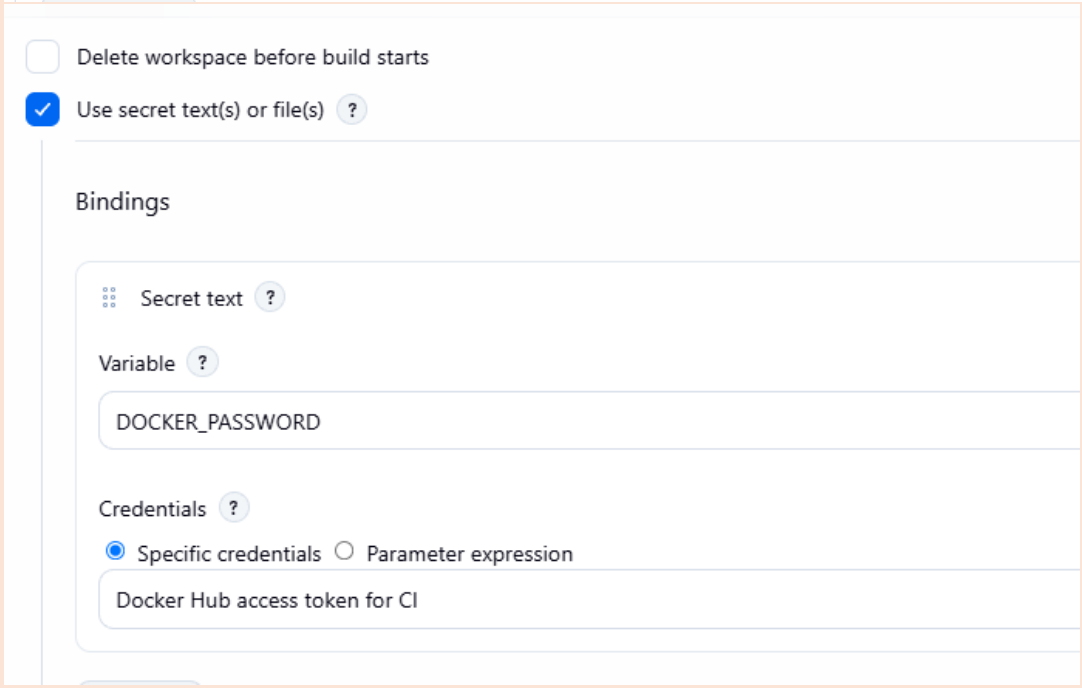
Expected flow:

Step 7: Verify the Image on Docker Hub

1. Go to **hub.docker.com/r/sunith18/sabear_simplecutomerapp**
2. Click the **Tags** tab
3. Confirm the tag matching the Jenkins build number is present

Screenshots:





Execute shell ?

Command

See the list of available environment variables

```
mvn clean install
```

Advanced

Execute shell ?

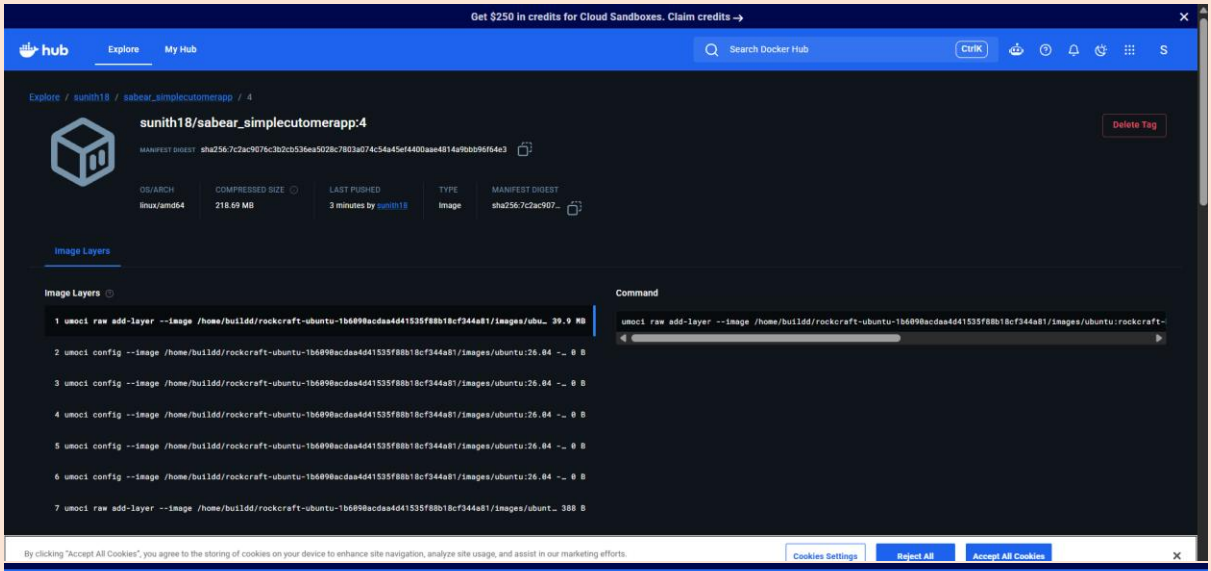
Command

See the list of available environment variables

```
chmod +x Dockerise.sh
./Dockerise.sh
```

```
#8 exporting to image
#8 exporting layers 0.0s done
#8 writing image sha256:9d33081f70c80bcf1f54aa037dfee623024a33e392df4918f30d728743813d02 done
#8 naming to docker.io/sunith18/sabear_simplecutomerapp:4 0.0s done
#8 DONE 0.0s
The push refers to repository [docker.io/sunith18/sabear_simplecutomerapp]
9e0a1c51ca4f: Preparing
5f70bf18a086: Preparing
5f70bf18a086: Preparing
fd905c67abb3: Preparing
5f70bf18a086: Preparing
a1798d505ce2: Preparing
b6e4ccb94c15: Preparing
1d7cf04deb8e: Preparing
9dc6fe2369f3: Preparing
343d675b5d90: Preparing
f98d83061adc: Preparing
f6e0e88dd8e1: Preparing
1d7cf04deb8e: Waiting
9dc6fe2369f3: Waiting
343d675b5d90: Waiting
f98d83061adc: Waiting
f6e0e88dd8e1: Waiting
b6e4ccb94c15: Mounted from library/tomcat
5f70bf18a086: Mounted from library/tomcat
a1798d505ce2: Mounted from library/tomcat
fd905c67abb3: Mounted from library/tomcat
9dc6fe2369f3: Mounted from library/tomcat
343d675b5d90: Mounted from library/tomcat
1d7cf04deb8e: Mounted from library/tomcat
f98d83061adc: Mounted from library/tomcat
f6e0e88dd8e1: Mounted from library/tomcat
9e0a1c51ca4f: Pushed
4: digest: sha256:7c2ac9076c3b2cb536ea5028c7803a074c54a45ef4400aae4814a9bbb96f64e3 size: 2822
Finished: SUCCESS
```

Sunith-Jenkins-Article



hub
Explore
My Hub
Explore / sunith18 / sabear_simplecutomerapp / 4
sunith18/sabear_simplecutomerapp:4
MANIFEST DIGEST sha256:7c2ac9076c3b2cb536ea5028c7803a074c54a45ef4400aae4814a9bbb96f64e3
OS/ARCH
linux/amd64
COMPRESSED SIZE
218.69 MB
LAST PUSHED
3 minutes by sunith18
TYPE
Image
MANIFEST DIGEST
sha256:7c2ac907…
Image Layers
Image Layers
1 umoci raw add-layer --image /home/buildd/rockcraft-ubuntu-1b6090acdaa4d41535f88b18cf344a81/images/ubu… 39.9 MB
2 umoci config --image /home/buildd/rockcraft-ubuntu-1b6090acdaa4d41535f88b18cf344a81/images/ubuntu:26.04 -… 0 B
3 umoci config --image /home/buildd/rockcraft-ubuntu-1b6090acdaa4d41535f88b18cf344a81/images/ubuntu:26.04 -… 0 B
4 umoci config --image /home/buildd/rockcraft-ubuntu-1b6090acdaa4d41535f88b18cf344a81/images/ubuntu:26.04 -… 0 B
5 umoci config --image /home/buildd/rockcraft-ubuntu-1b6090acdaa4d41535f88b18cf344a81/images/ubuntu:26.04 -… 0 B
6 umoci config --image /home/buildd/rockcraft-ubuntu-1b6090acdaa4d41535f88b18cf344a81/images/ubuntu:26.04 -… 0 B
7 umoci raw add-layer --image /home/buildd/rockcraft-ubuntu-1b6090acdaa4d41535f88b18cf344a81/images/ubunt… 388 B